

William Stallings Computer Organization and Architecture 10th Edition (Chp 3 & 4

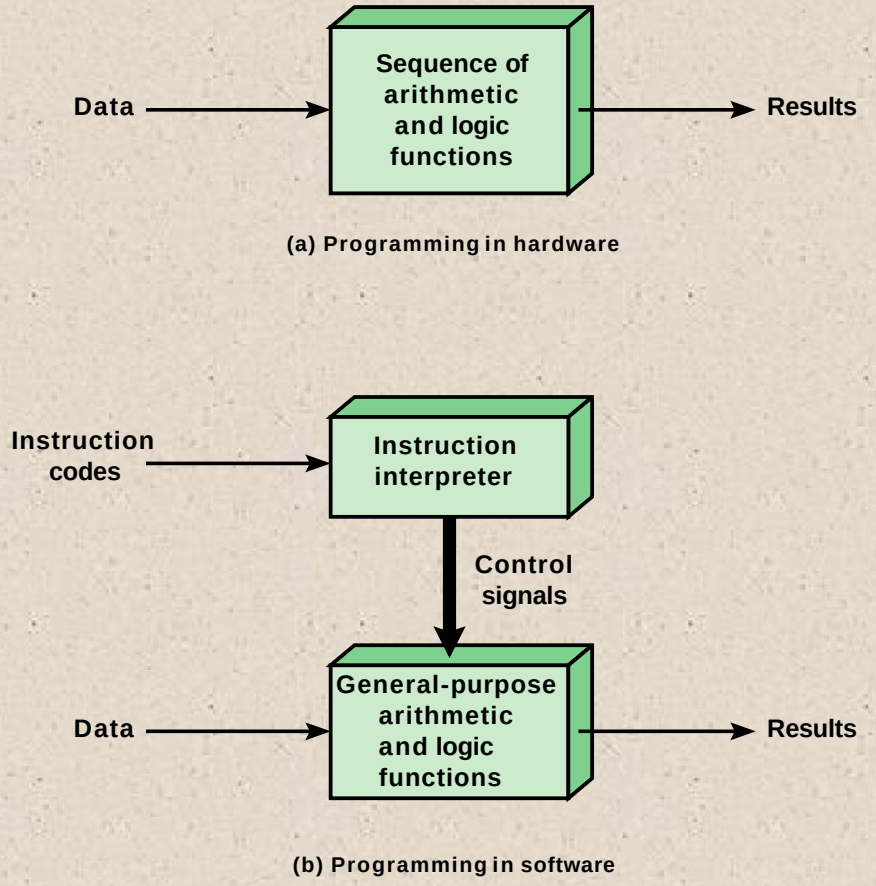
© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Computer Components

- Contemporary computer designs are based on concepts developed by John von Neumann at the Institute for Advanced Studies, Princeton
- Referred to as the *von Neumann architecture* and is based on three key concepts:
  - Data and instructions are stored in a single read-write memory
  - The contents of this memory are addressable by location, without regard to the type of data contained there
  - Execution occurs in a sequential fashion (unless explicitly modified) from one instruction to the next
- *Hardwired program*
  - The result of the process of **connecting** the **various components** in the desired configuration

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Hardware and Software Approaches



Suppose we construct a general-purpose configuration of arithmetic and logic functions. This set of hardware will perform various functions on data depending on control signals applied to the hardware. In the original case of customized hardware, the system accepts data and produces results (Figure 3.1a). With general-purpose hardware, the system accepts data and control signals and produces results. Thus, instead of rewiring the hardware for each new program, the programmer merely needs to supply a new set of control signals.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Software

- A sequence of codes or instructions
- Part of the hardware interprets each instruction and generates control signals
- Provide a new sequence of codes for each new program instead of rewiring the hardware

Major components:

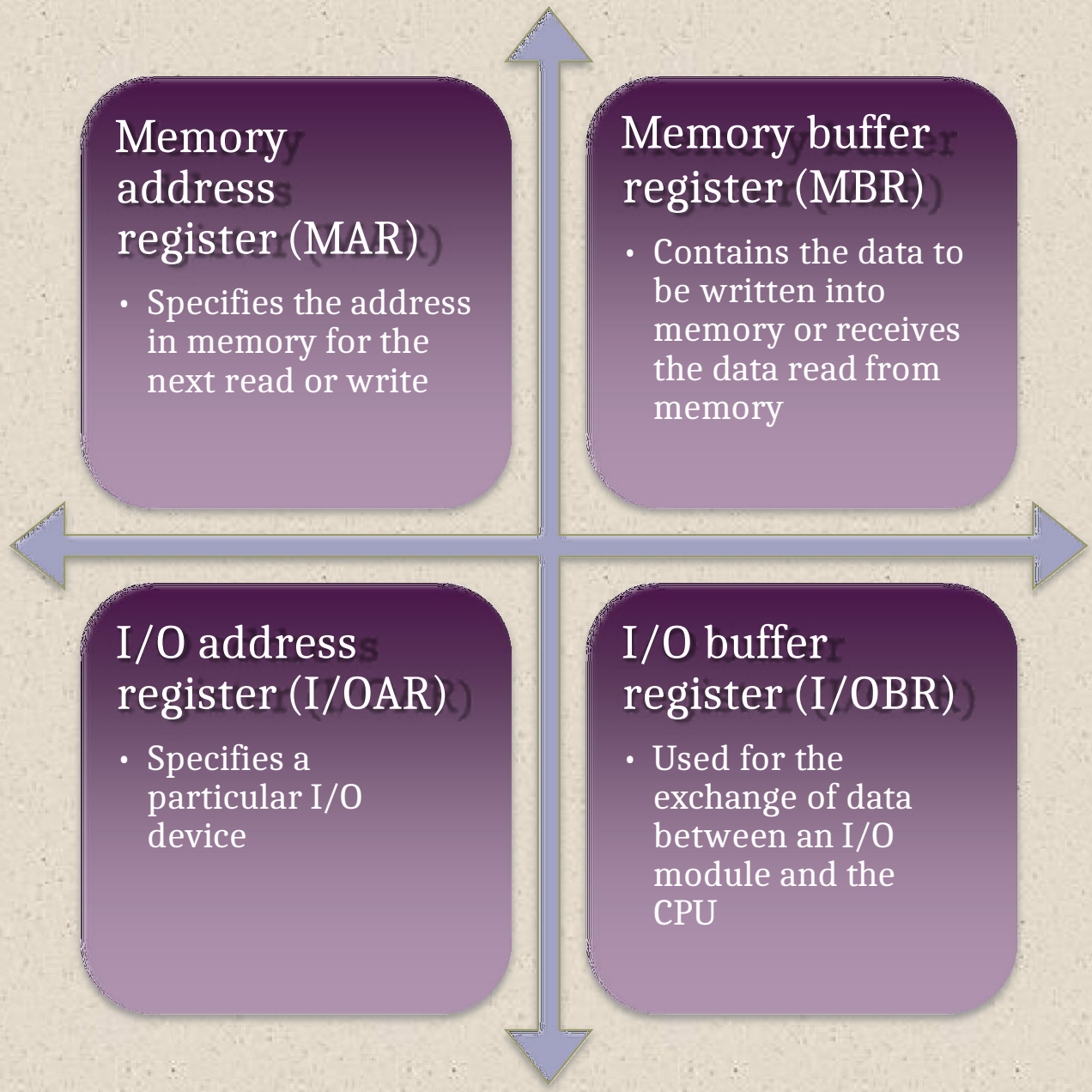
- CPU
  - Instruction interpreter
  - Module of general-purpose arithmetic and logic functions
- I/O Components
  - Input module
    - Contains basic components for accepting data and instructions and converting them into an internal form of signals usable by the system
  - Output module
    - Means of reporting results

Software

I/O Components



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



MEMORY

MAR

MBR

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

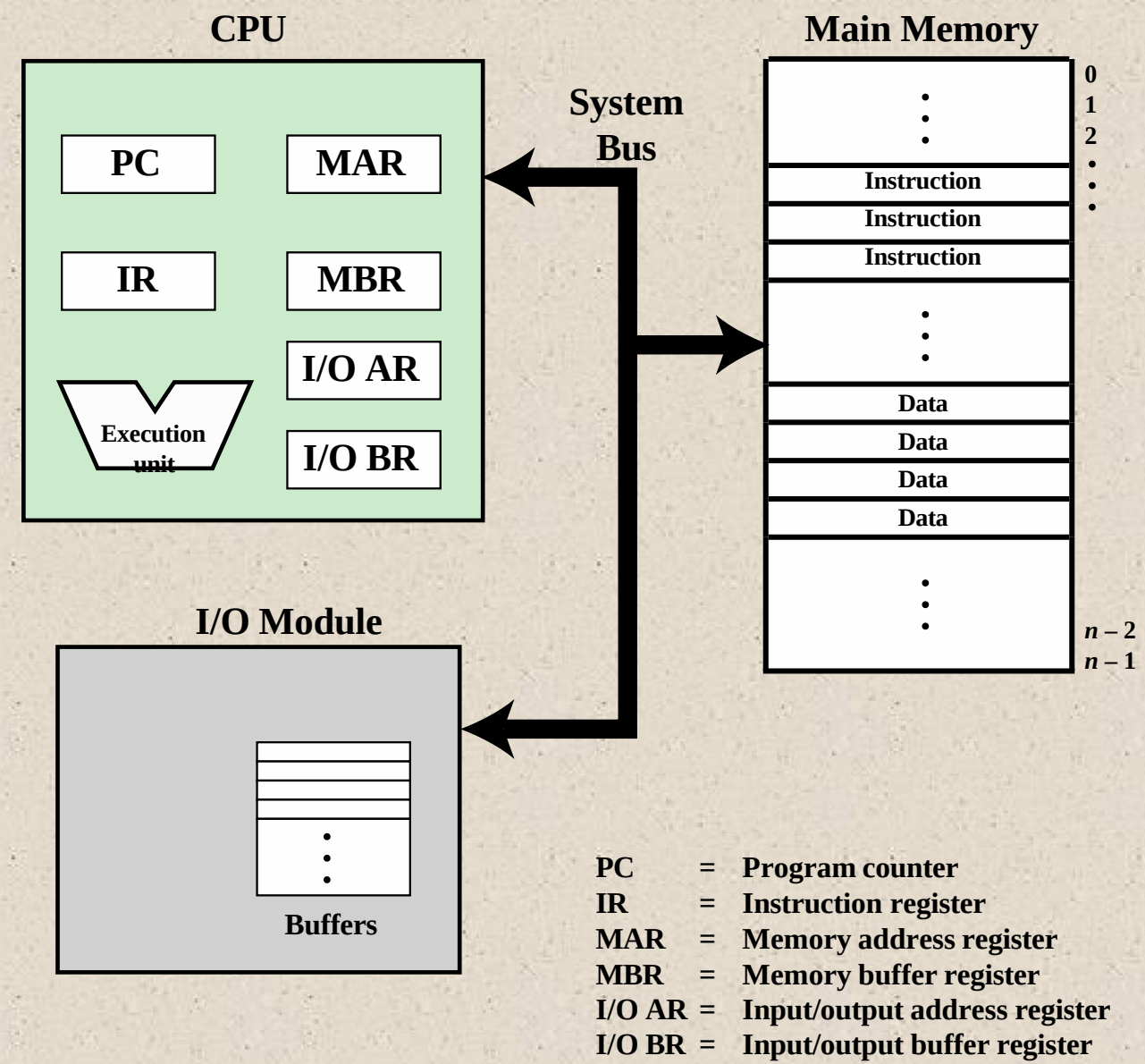


Figure 3.2 Computer Components: Top-Level View

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

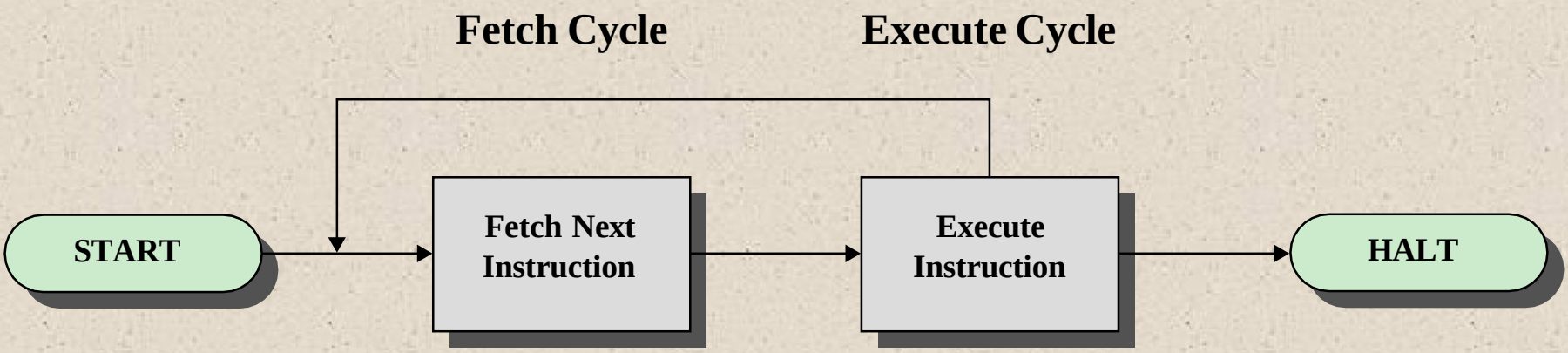


Figure 3.3 Basic Instruction Cycle

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

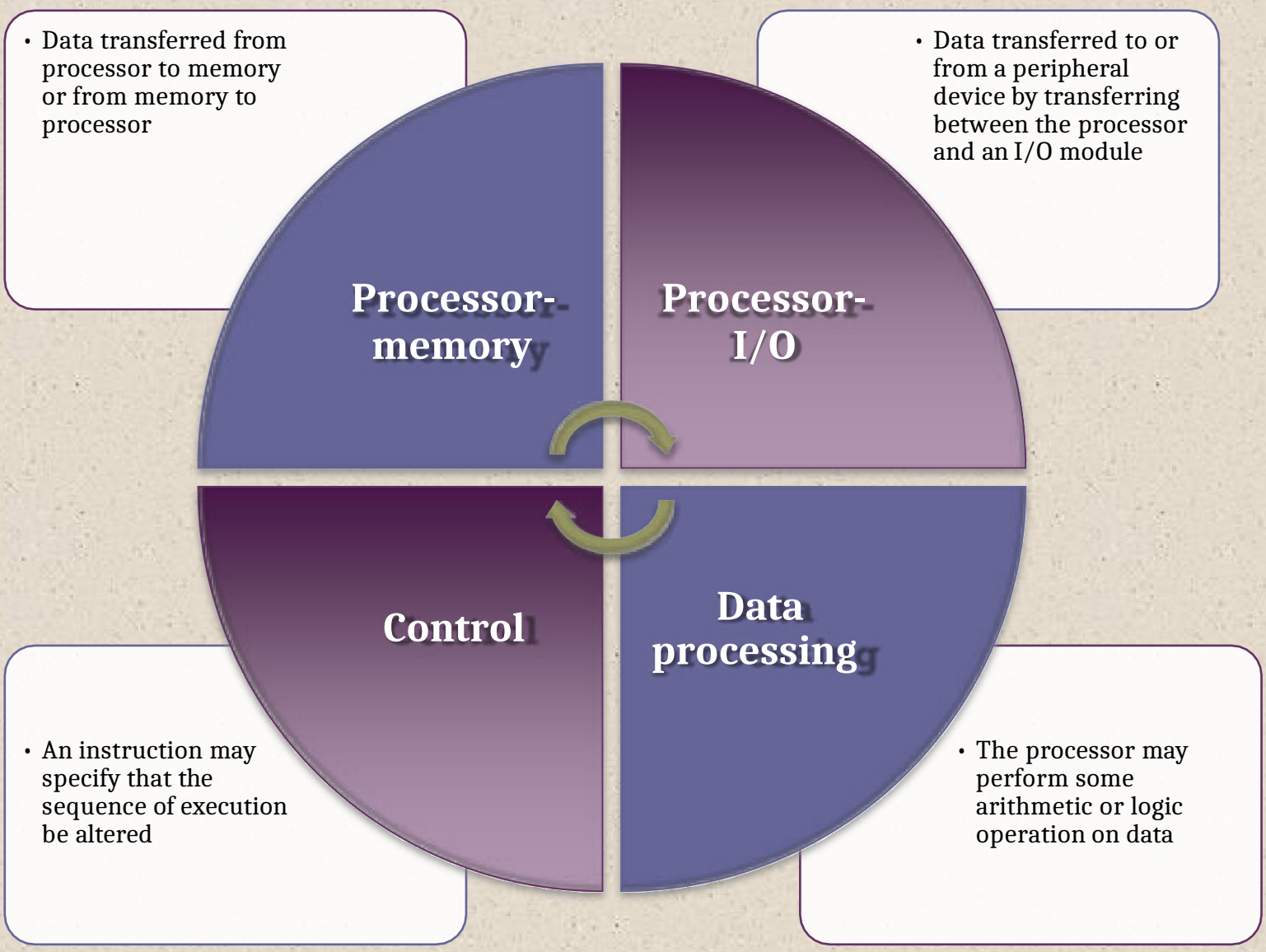
Fetch Cycle

- At the beginning of each instruction cycle the processor fetches an instruction from memory
- The program counter (PC) holds the address of the instruction to be fetched next
- The processor increments the PC after each instruction fetch so that it will fetch the next instruction in sequence
- The fetched instruction is loaded into the instruction register (IR)
- The processor interprets the instruction and performs the required action

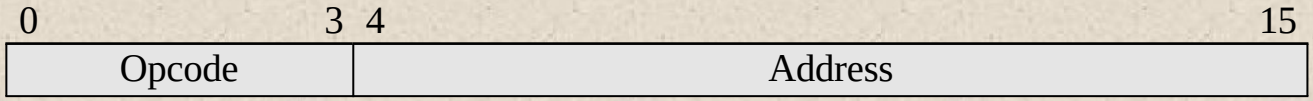


© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Action Categories



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) Instruction format



(b) Integer format

Program Counter (PC) = Address of instruction
Instruction Register (IR) = Instruction being executed
Accumulator (AC) = Temporary storage

(c) Internal CPU registers

0001 = Load AC from Memory
0010 = Store AC to Memory
0101 = Add to AC from Memory

(d) Partial list of opcodes

Figure 3.4 Characteristics of a Hypothetical Machine

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

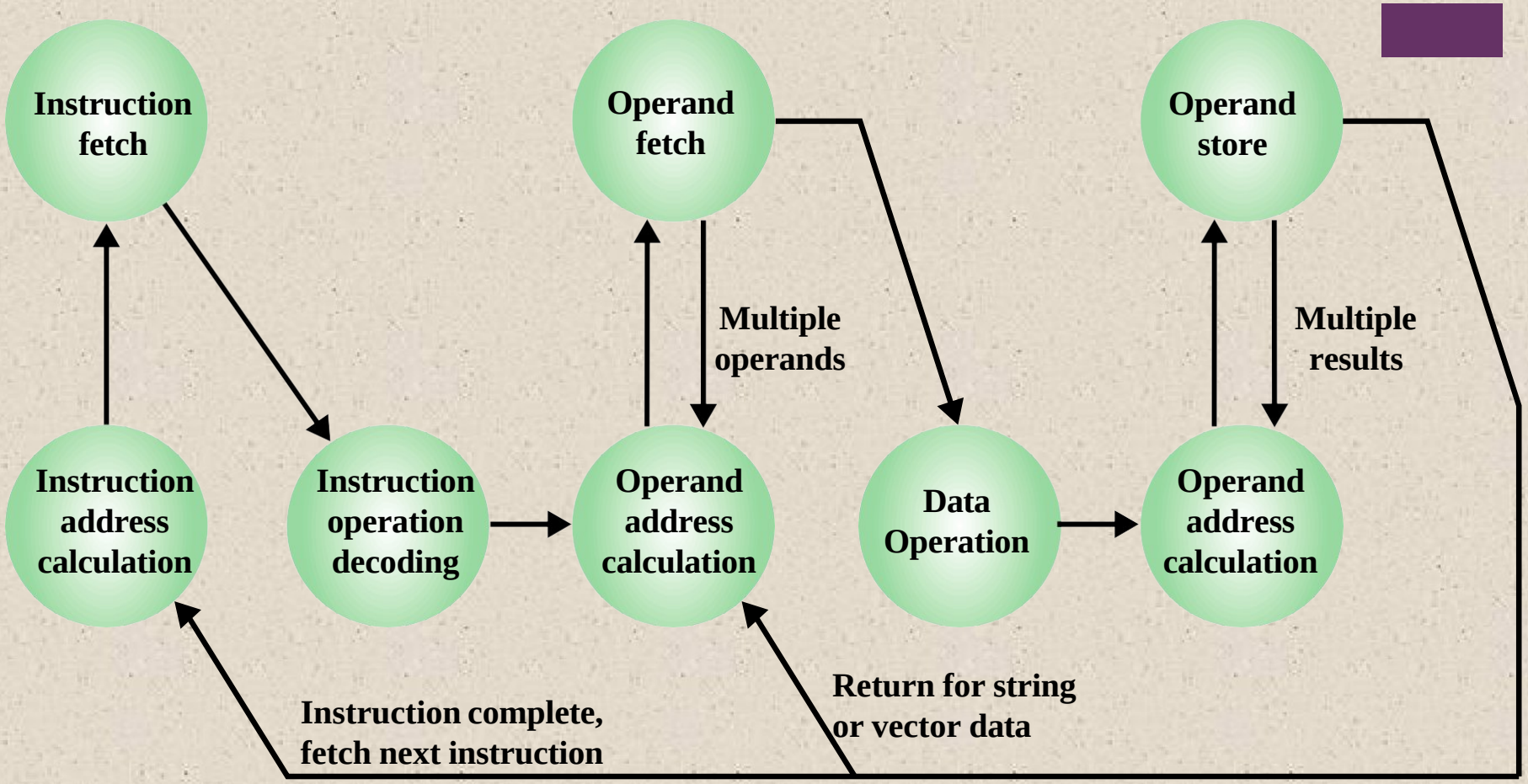


Figure 3.6 Instruction Cycle State Diagram

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Interrupt

- The interrupt is a signal emitted by hardware or software when a process or an event needs immediate attention. It alerts the processor to a high-priority process requiring interruption of the current working process. In I/O devices one of the bus control lines is dedicated for this purpose and is called the *Interrupt Service Routine (ISR)*.

- When a device raises an interrupt at let's say process i, the processor first completes the execution of instruction i. Then it loads the Program Counter (PC) with the address of the first instruction of the ISR. Before loading the Program Counter with the address, the address of the interrupted instruction is moved to a temporary location. Therefore, after handling the interrupt the processor can continue with process i+1.

- While the processor is handling the interrupts, it must inform the device that its request has been recognized so that it stops sending the interrupt request signal. Also, saving the registers so that the interrupted process can be restored in the future, increases the delay between the time an interrupt is received and the start of the execution of the ISR. This is called Interrupt Latency.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Type of interrupt

- **Software Interrupts:** A sort of interrupt called a software interrupt is one that is produced by software or a system as opposed to hardware. Traps and exceptions are other names for software interruptions. They serve as a signal for the operating system or a system service to carry out a certain function or respond to an error condition. A particular instruction known as a "interrupt instruction" is used to create software interrupts. When the interrupt instruction is used, the processor stops what it is doing and switches over to a particular interrupt handler code. The interrupt handler routine completes the required work or handles any errors before handing back control to the interrupted application.

- **Hardware Interrupts:** In a hardware interrupt, all the devices are connected to the Interrupt Request Line. A single request line is used for all the n devices. To request an interrupt, a device closes its associated switch. When a device requests an interrupt, the value of INTR is the logical OR of the requests from individual devices.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation, request service from the processor, or to signal a variety of error conditions.
Hardware failure	Generated by a failure such as power failure or memory parity error.

Table 3.1

Classes of Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

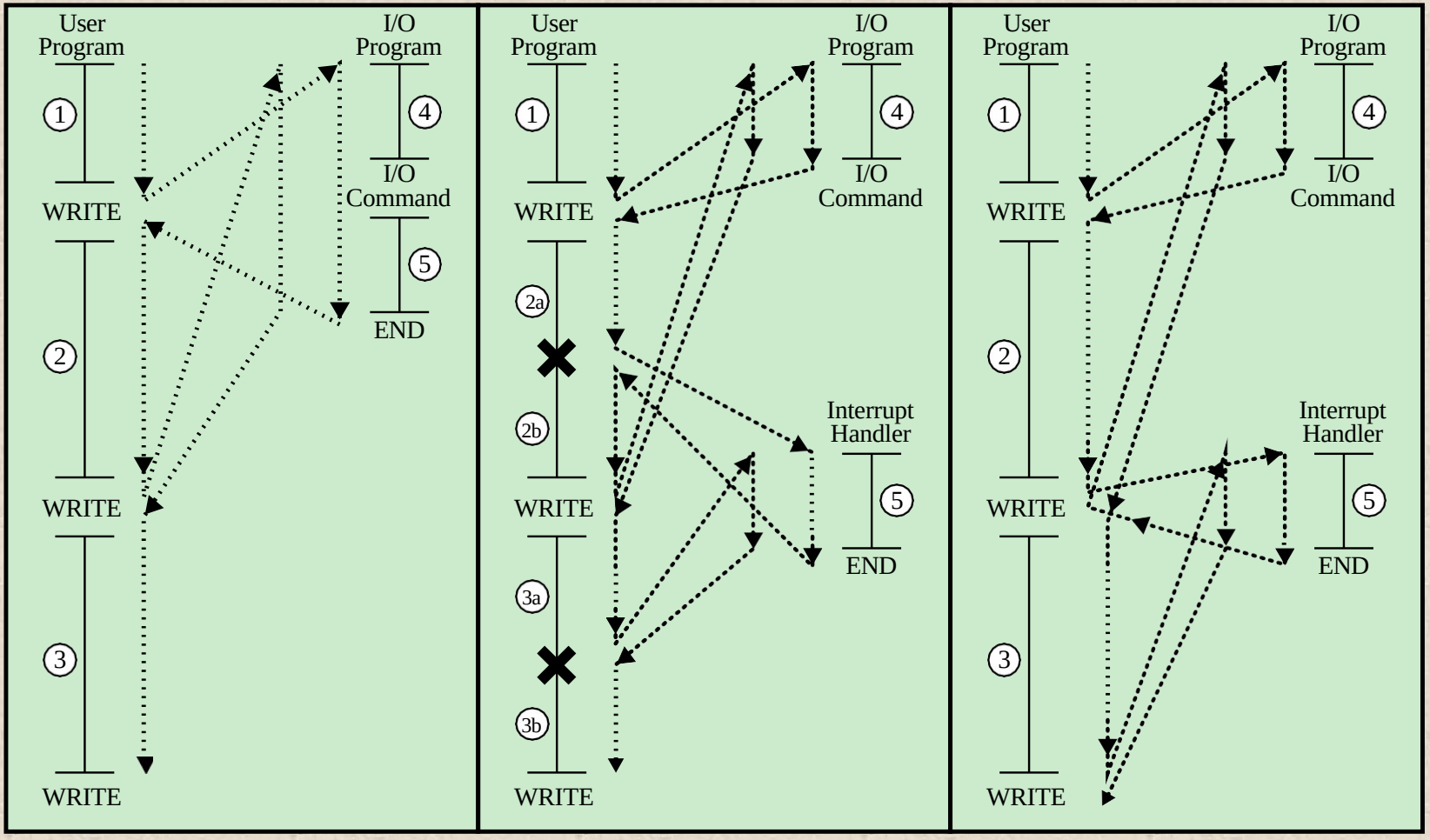


Figure 3.7 Program Flow of Control Without and With Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

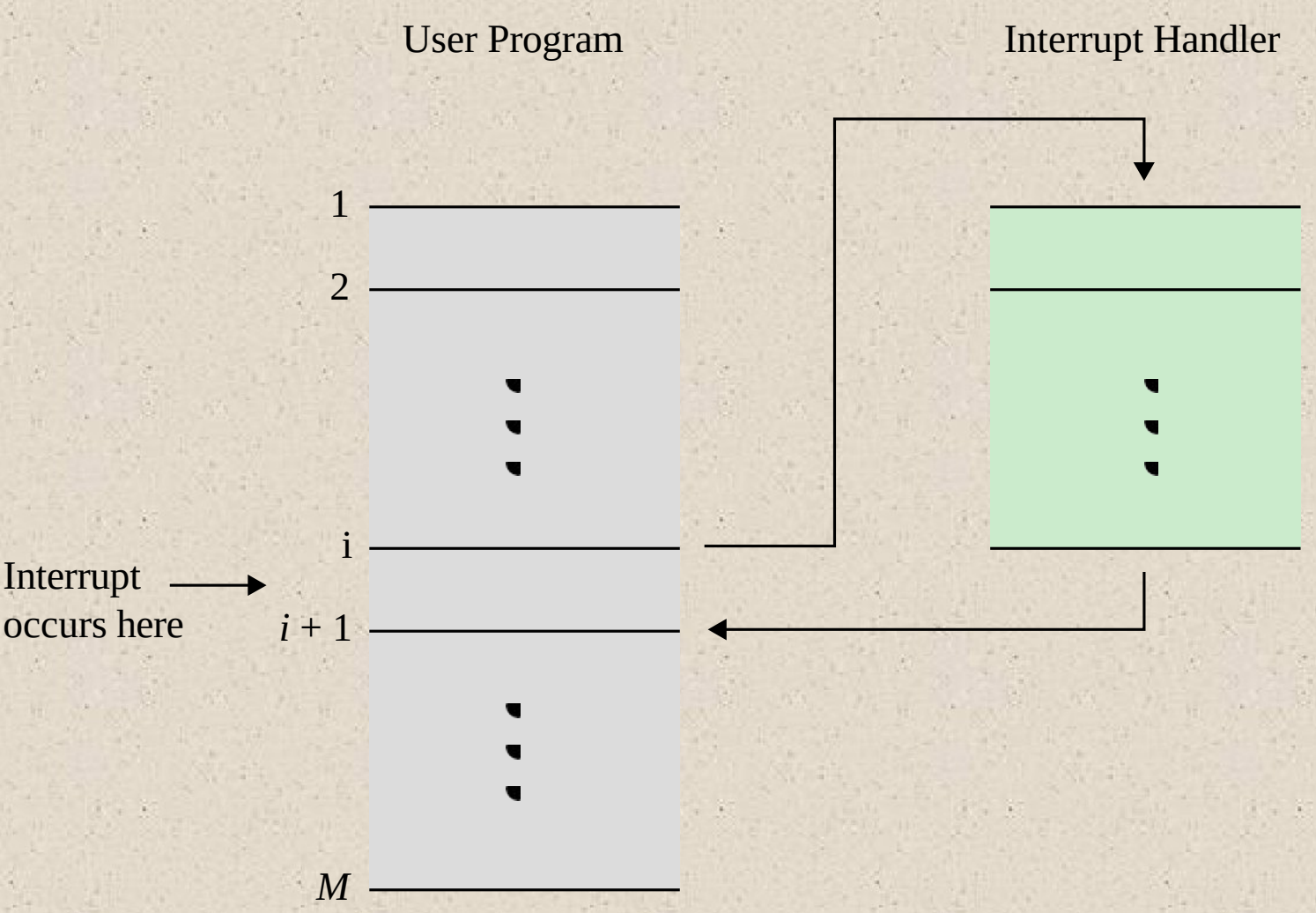


Figure 3.8 Transfer of Control via Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

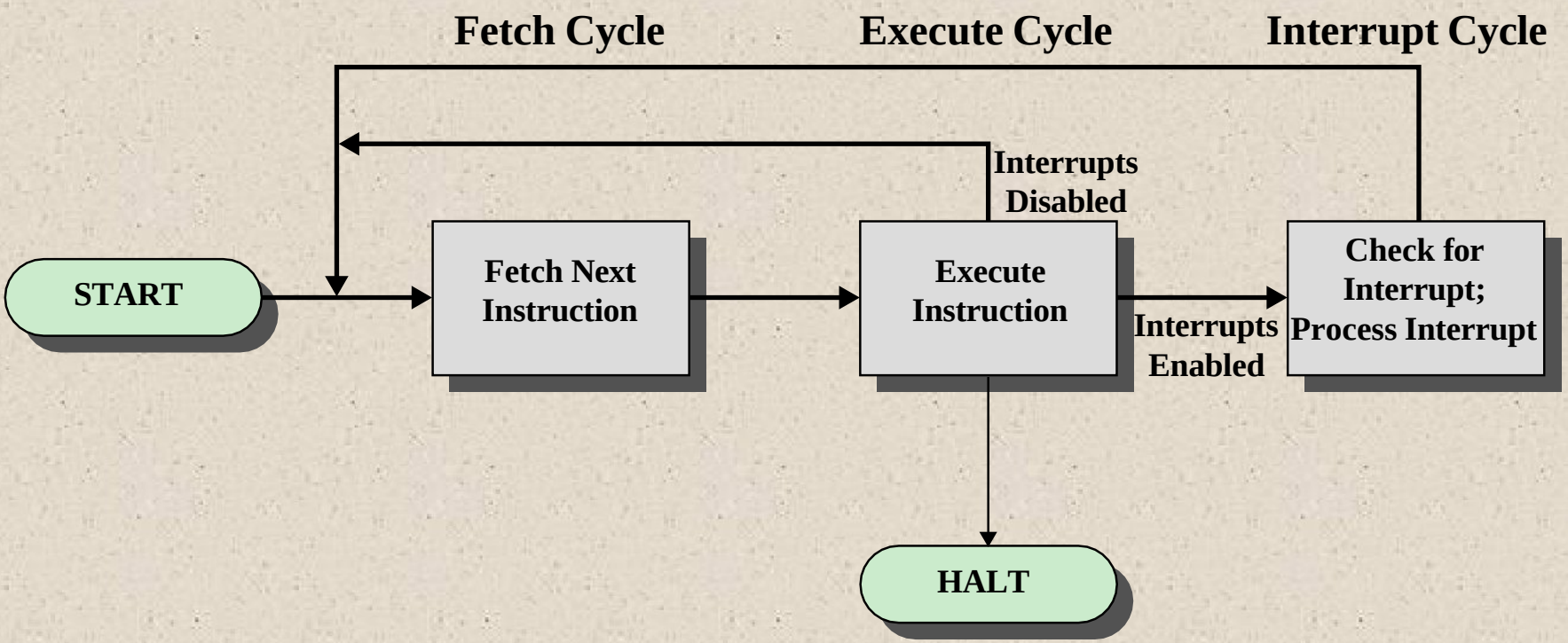


Figure 3.9 Instruction Cycle with Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

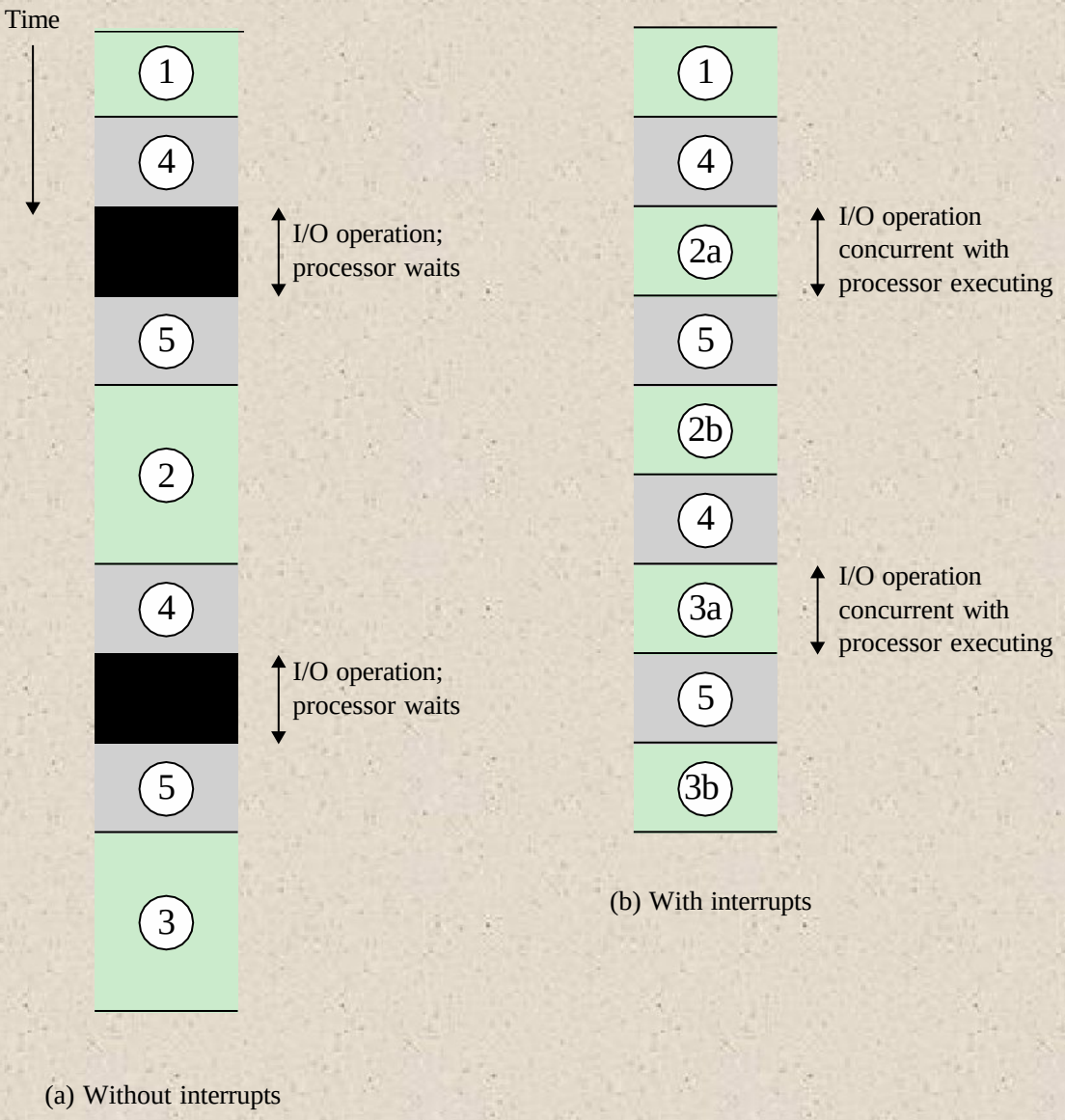


Figure 3.10 Program Timing: Short I/O Wait

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

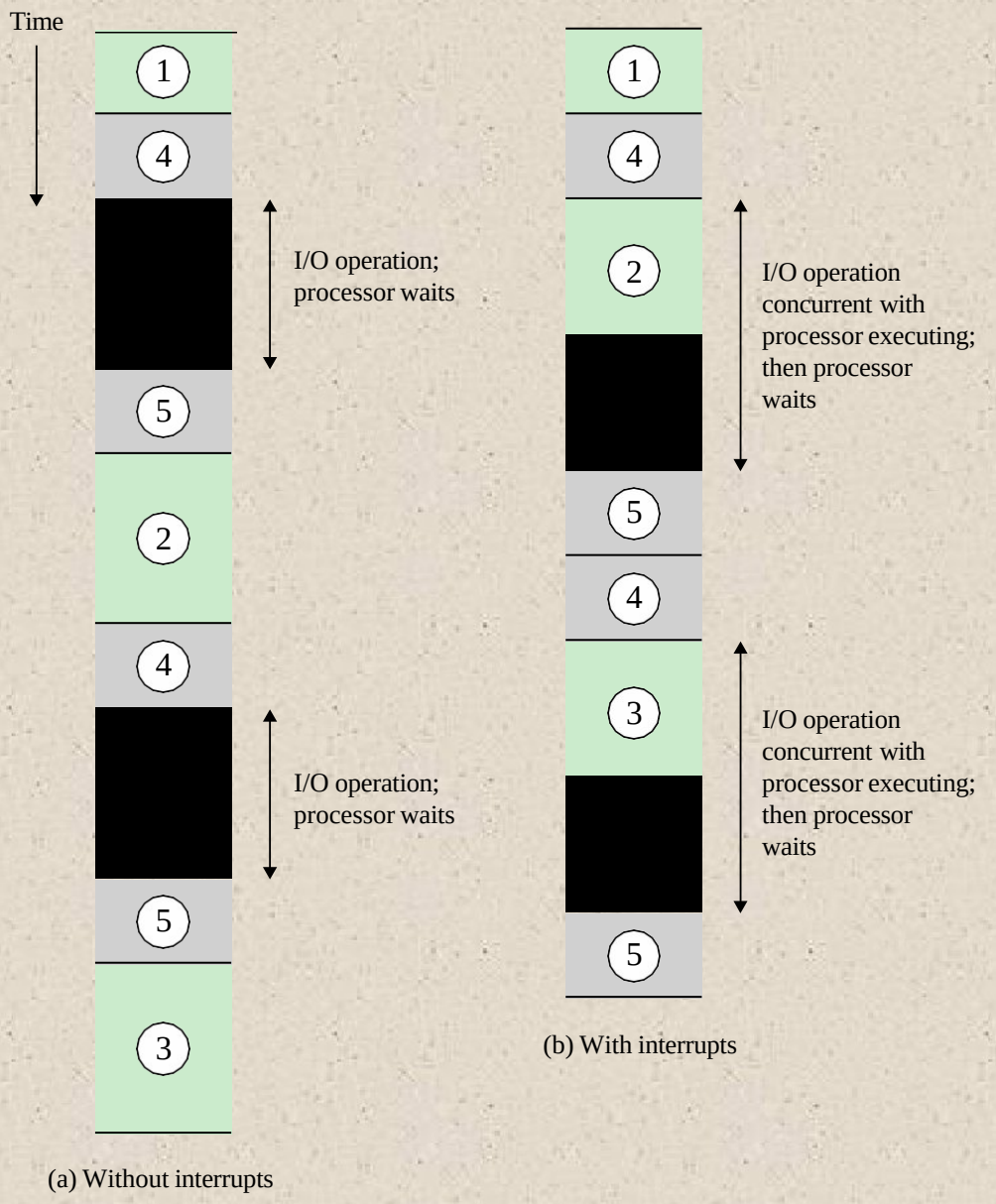


Figure 3.11 Program Timing: Long I/O Wait

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

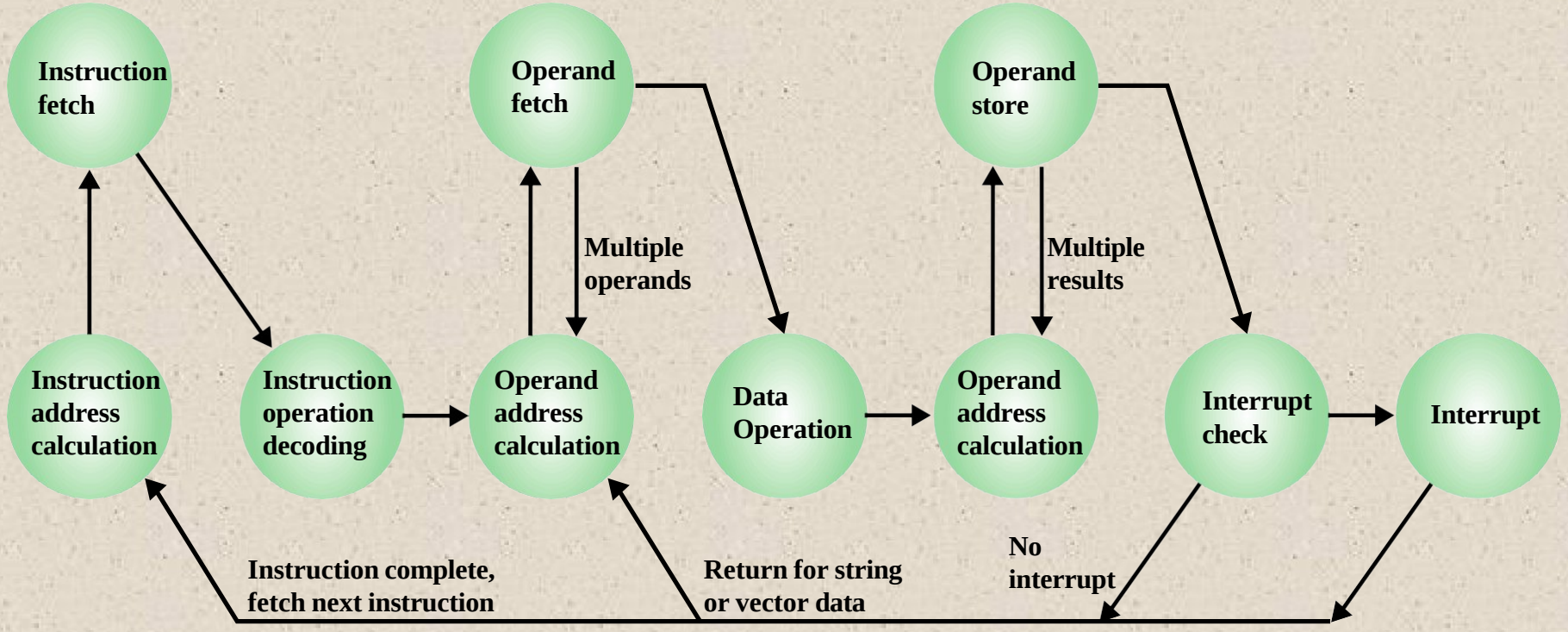


Figure 3.12 Instruction Cycle State Diagram, With Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Multiple Interrupts

- Disable interrupt

✓ processor will ignore further interrupts whilst processing one interrupt

✓ Interrupts remain pending and are checked after first interrupt has been processed

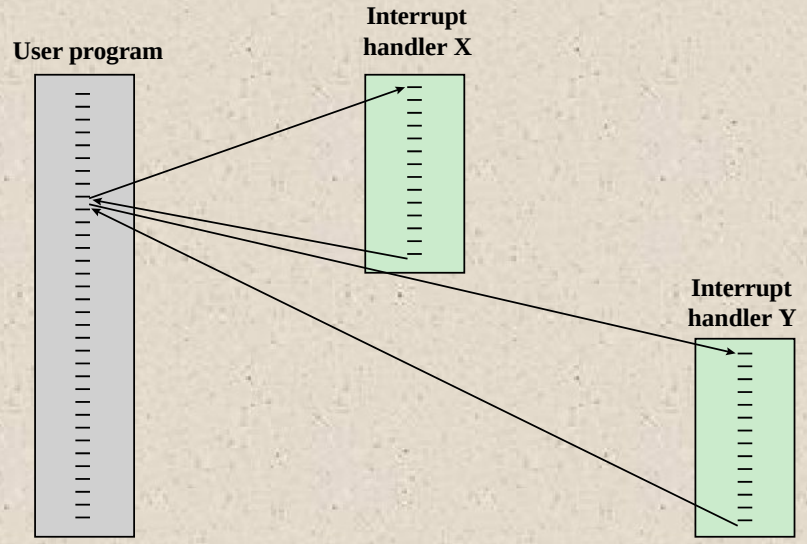
✓ Interrupts handled in sequence as they occur

- Define priorities

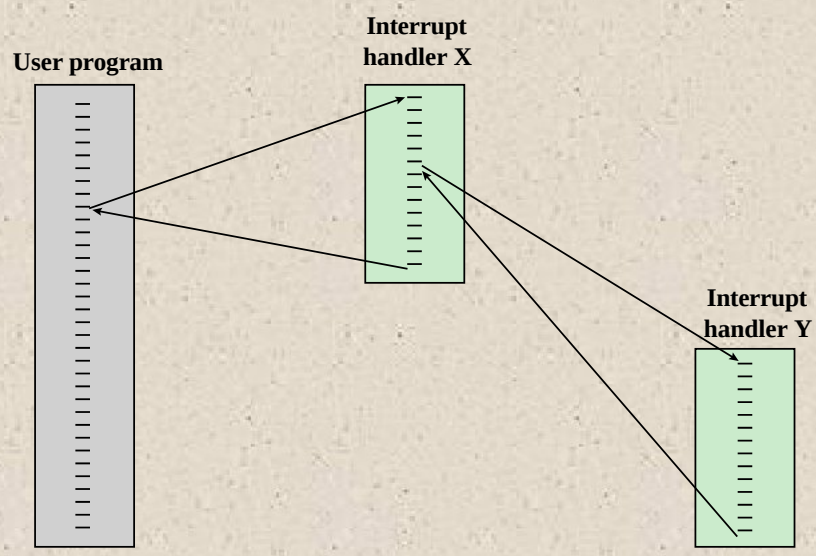
✓ Low priority interrupts can be interrupted by higher priority interrupts

✓ When higher priority interrupt has been processed processor returns to previous interrupt

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) Sequential interrupt processing



(b) Nested interrupt processing

Figure 3.13 Transfer of Control with Multiple Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

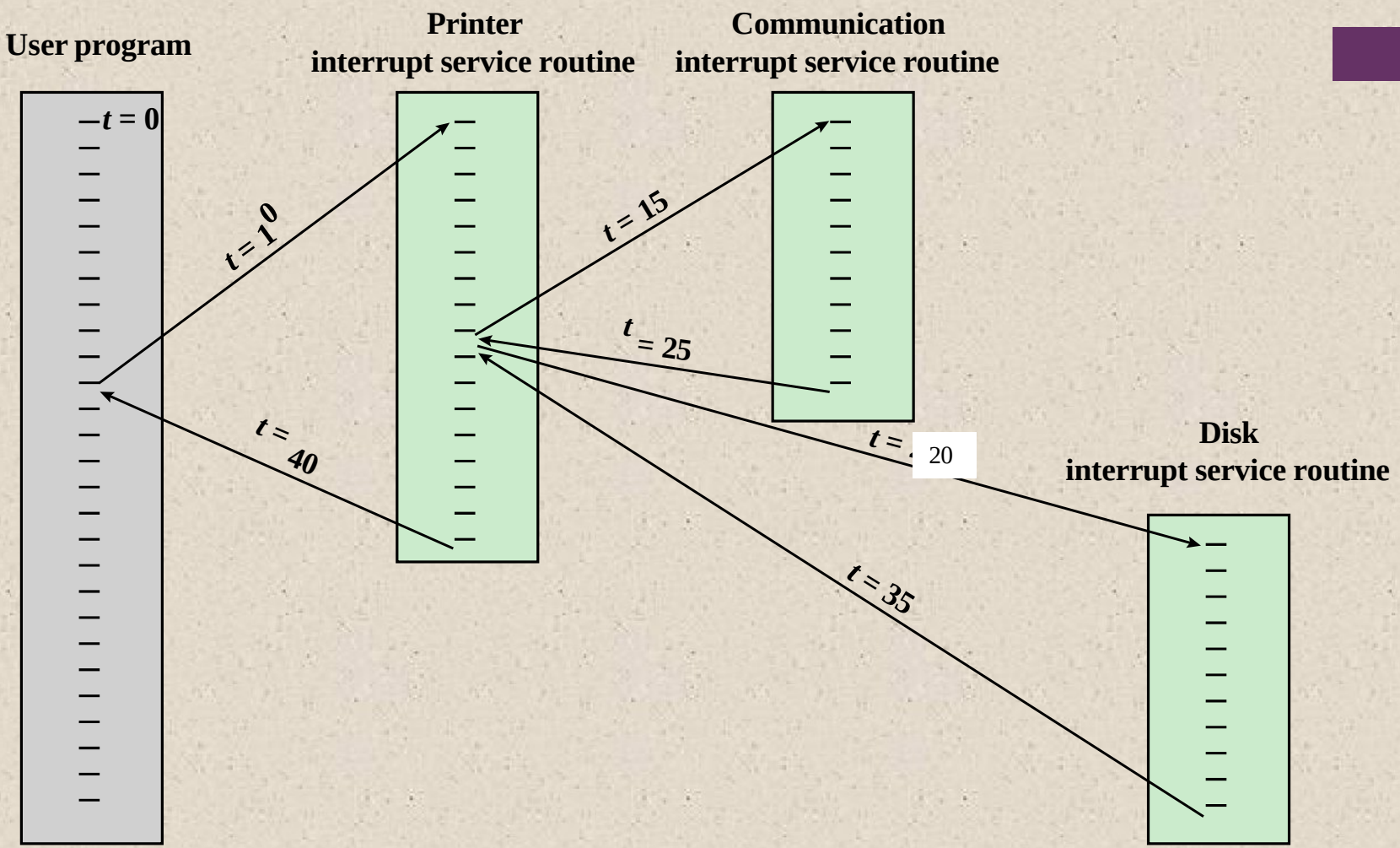


Figure 3.14 Example Time Sequence of Multiple Interrupts

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

I/O Function

- I/O module can exchange data directly with the processor
- Processor can read data from or write data to an I/O module
  - Processor identifies a specific device that is controlled by a particular I/O module
  - I/O instructions rather than memory referencing instructions
- In some cases it is desirable to allow I/O exchanges to occur directly with memory
  - The processor grants to an I/O module the authority to read from or write to memory so that the I/O memory transfer can occur without tying up the processor
  - The I/O module issues read or write commands to memory relieving the processor of responsibility for the exchange
  - This operation is known as direct memory access (DMA)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

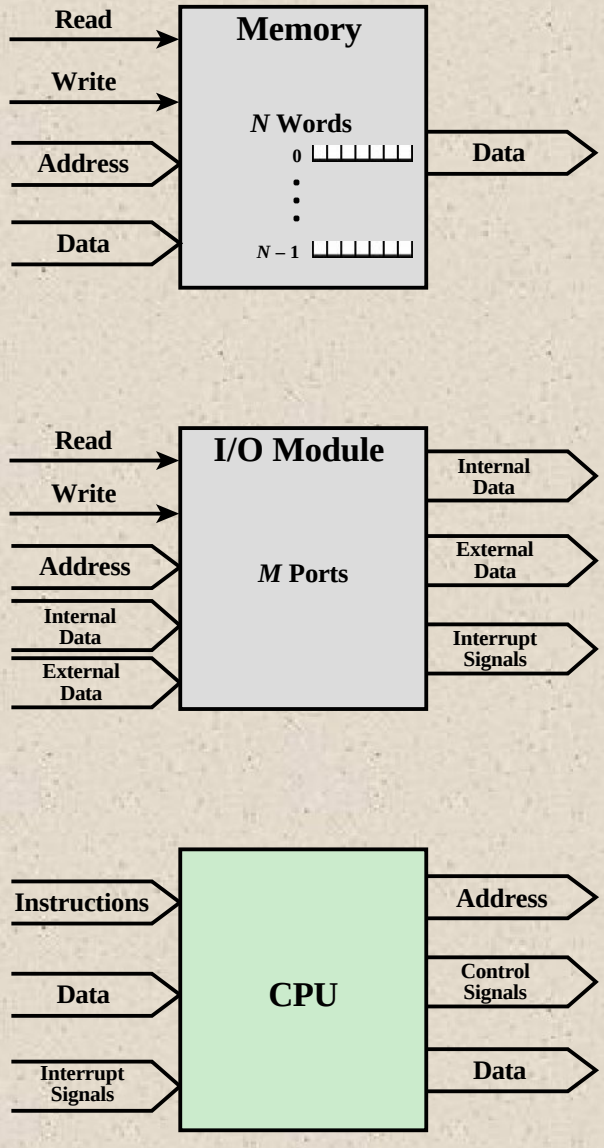


Figure 3.15 Computer Modules

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The interconnection structure must support the following types of transfers:

Memory to processor	Processor to memory	I/O to processor	Processor to I/O	I/O to or from memory
Processor reads an instruction or a unit of data from memory	Processor writes a unit of data to memory	Processor reads data from an I/O device via an I/O module	Processor sends data to the I/O device	An I/O module is allowed to exchange data directly with memory without going through the processor using direct memory access

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Bus Interconnection

A communication pathway connecting two or more devices

- Key characteristic is that it is a shared transmission medium

Signals transmitted by any one device are available for reception by all other devices attached to the bus

- If two devices transmit during the same time period their signals will overlap and become garbled



Typically consists of multiple communication lines

- Each line is capable of transmitting signals representing binary 1 and binary 0

Computer systems contain a number of different buses that provide pathways between components at various levels of the computer system hierarchy



System bus

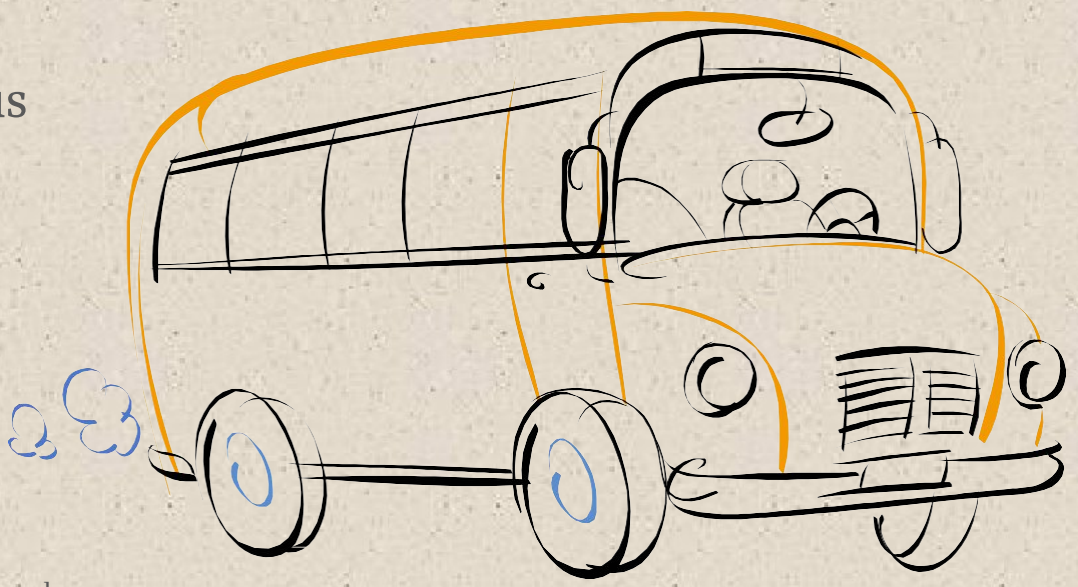
- A bus that connects major computer components (processor, memory, I/O)

The most common computer interconnection structures are based on the use of one or more system buses

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

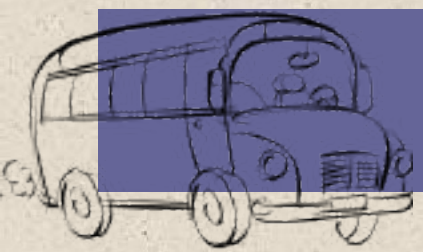
Data Bus

- Data lines that provide a path for moving data among system modules
- May consist of 32, 64, 128, or more separate lines
- The number of lines is referred to as the *width* of the data bus
- The number of lines determines how many bits can be transferred at a time
- The width of the data bus is a key factor in determining overall system performance



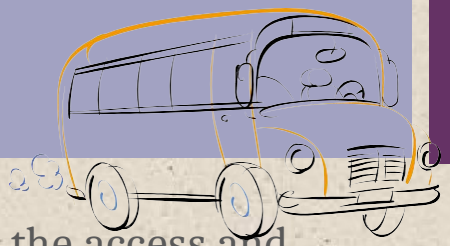
© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Address Bus



- Used to designate the source or destination of the data on the data bus
  - If the processor wishes to read a word of data from memory it puts the address of the desired word on the address lines
- Width determines the maximum possible memory capacity of the system
- Also used to address I/O ports
  - The higher order bits are used to select a particular module on the bus and the lower order bits select a memory location or I/O port within the module

Control Bus



- Used to control the access and the use of the data and address lines
- Because the data and address lines are shared by all components there must be a means of controlling their use
- Control signals transmit both command and timing information among system modules
- Timing signals indicate the validity of data and address information
- Command signals specify operations to be performed

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

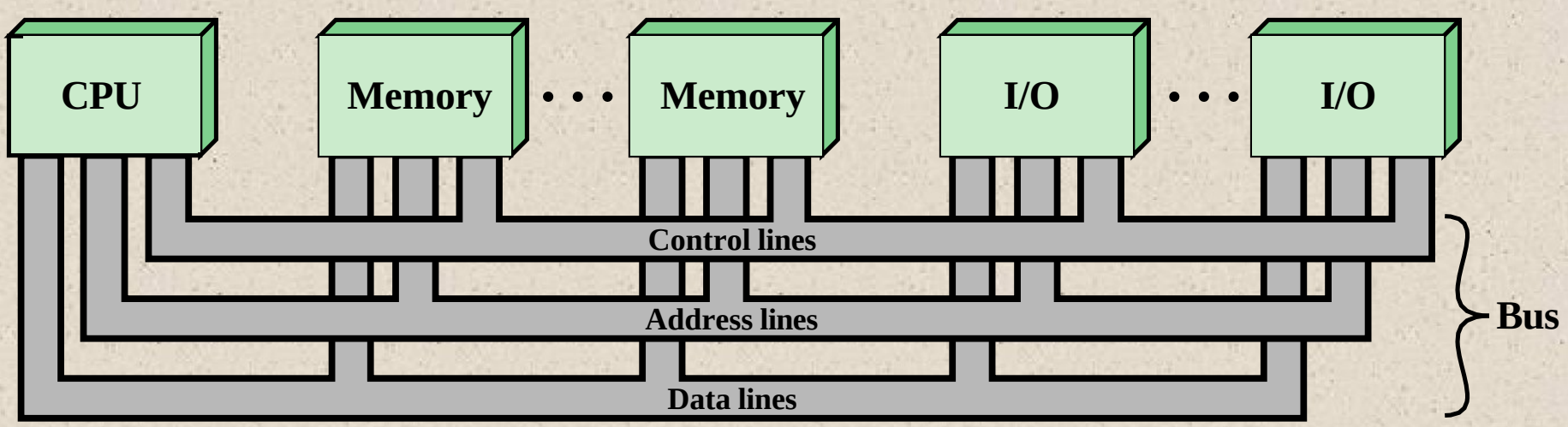


Figure 3.16 Bus Interconnection Scheme

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Point-to-Point Interconnect

Principal reason for change was the electrical constraints encountered with increasing the frequency of wide synchronous buses

At higher and higher data rates it becomes increasingly difficult to perform the synchronization and arbitration functions in a timely fashion

A conventional shared bus on the same chip magnified the difficulties of increasing bus data rate and reducing bus latency to keep up with the processors

Has lower latency, higher data rate, and better scalability

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Quick Path Interconnect

- Introduced in 2008
- Multiple direct connections
  - Direct pairwise connections to other components eliminating the need for arbitration found in shared transmission systems
- Layered protocol architecture
  - These processor level interconnects use a layered protocol architecture rather than the simple use of control signals found in shared bus arrangements
- Packetized data transfer
  - Data are sent as a sequence of packets each of which includes control headers and error control codes

QPI



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

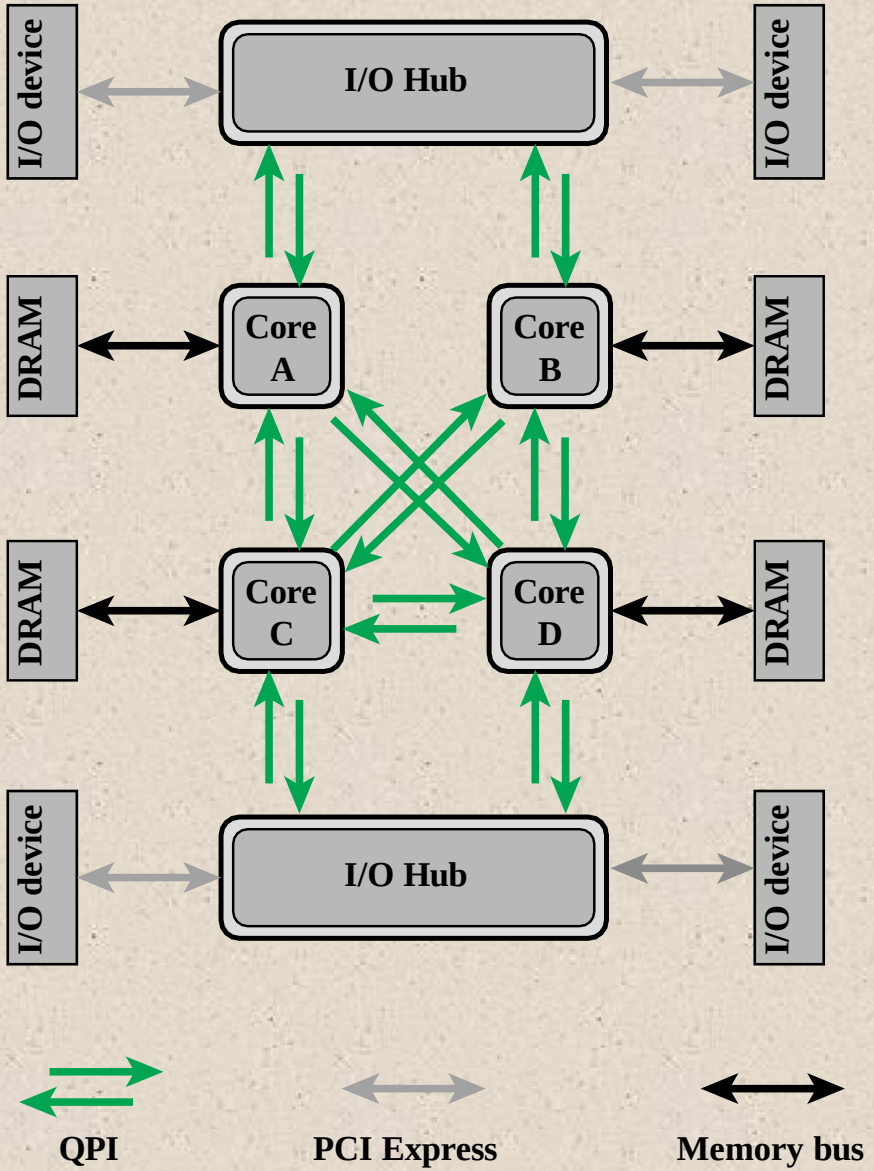


Figure 3.17 Multicore Configuration Using QPI

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

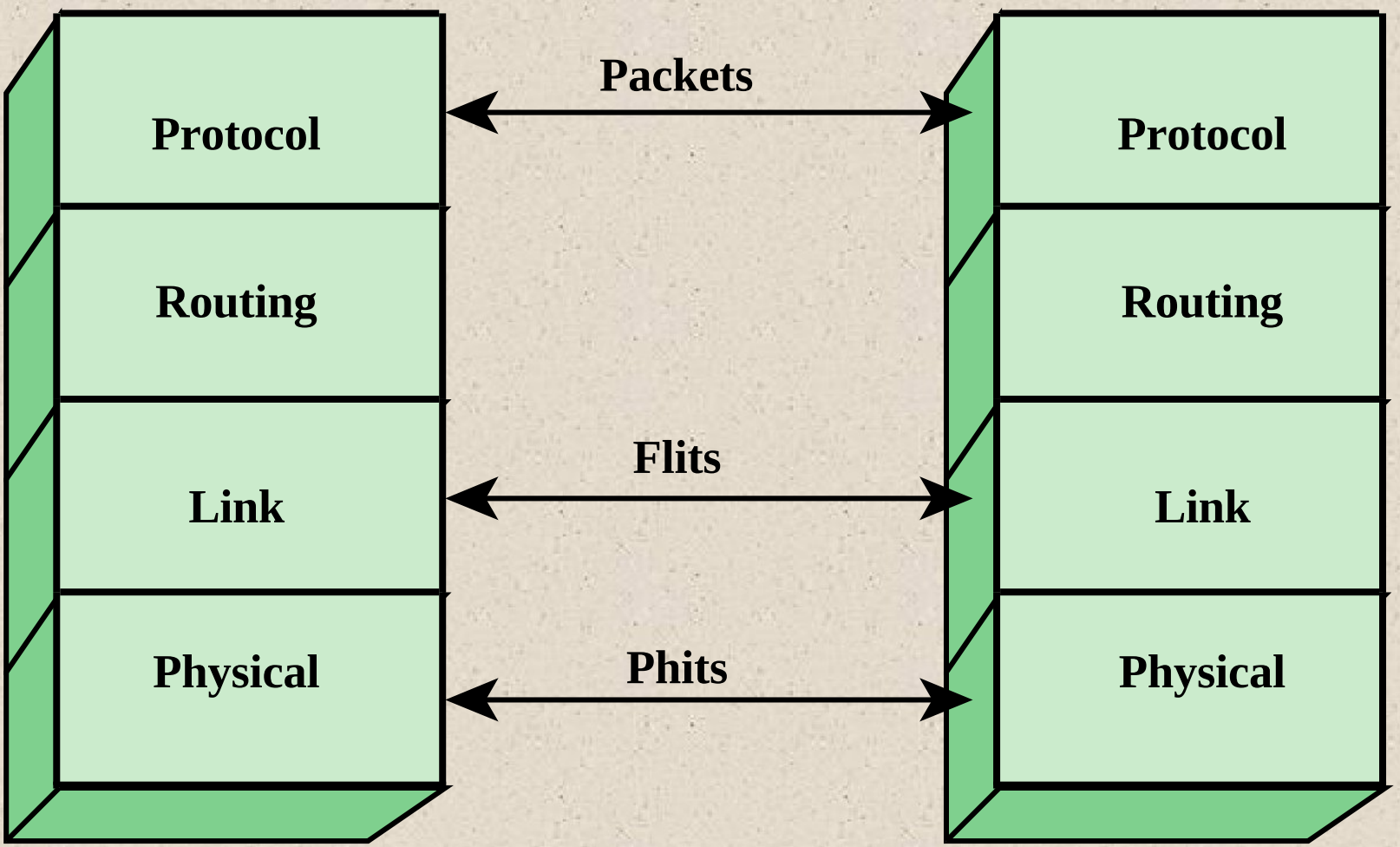


Figure 3.18 QPI Layers

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

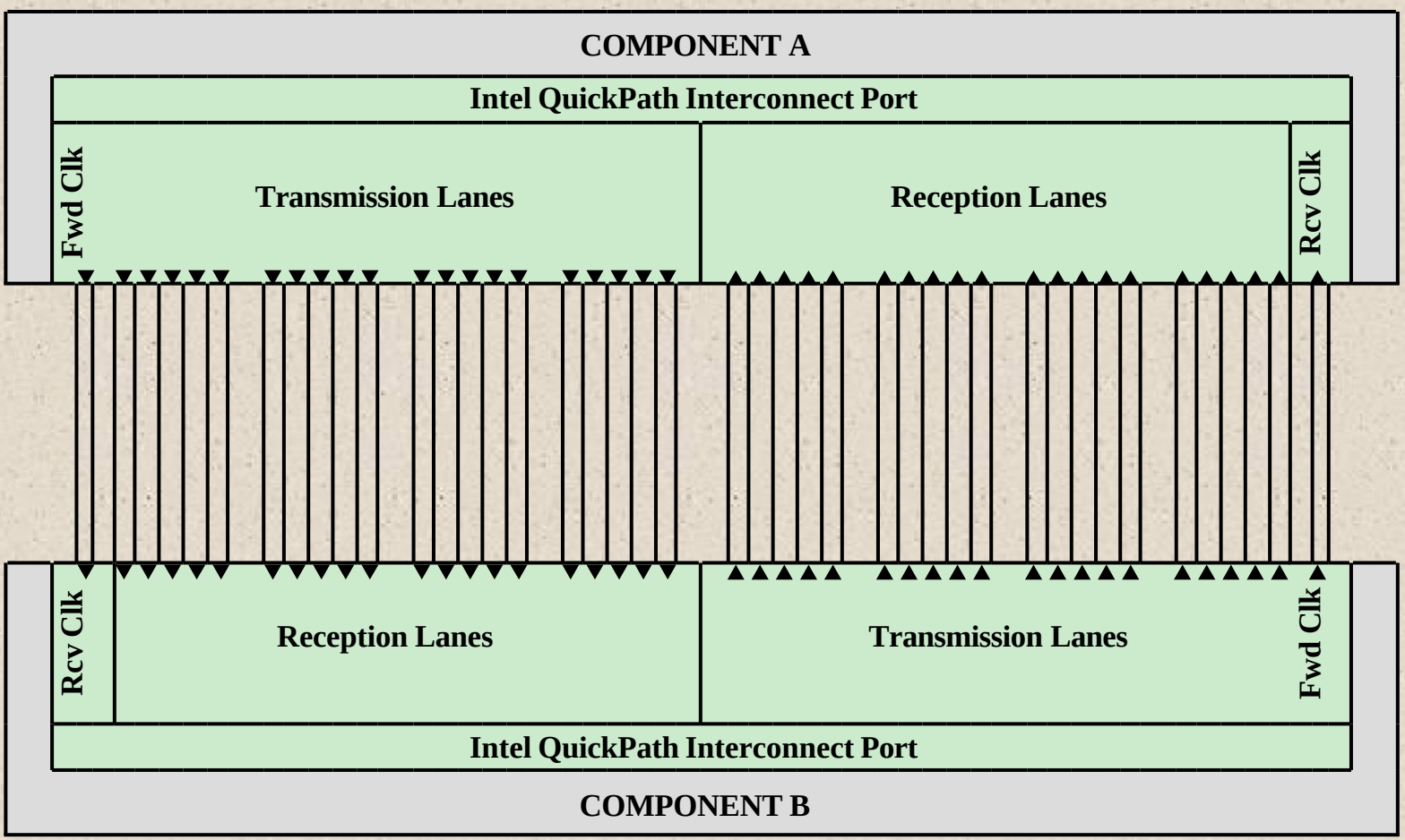


Figure 3.19 Physical Interface of the Intel QPI Interconnect

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

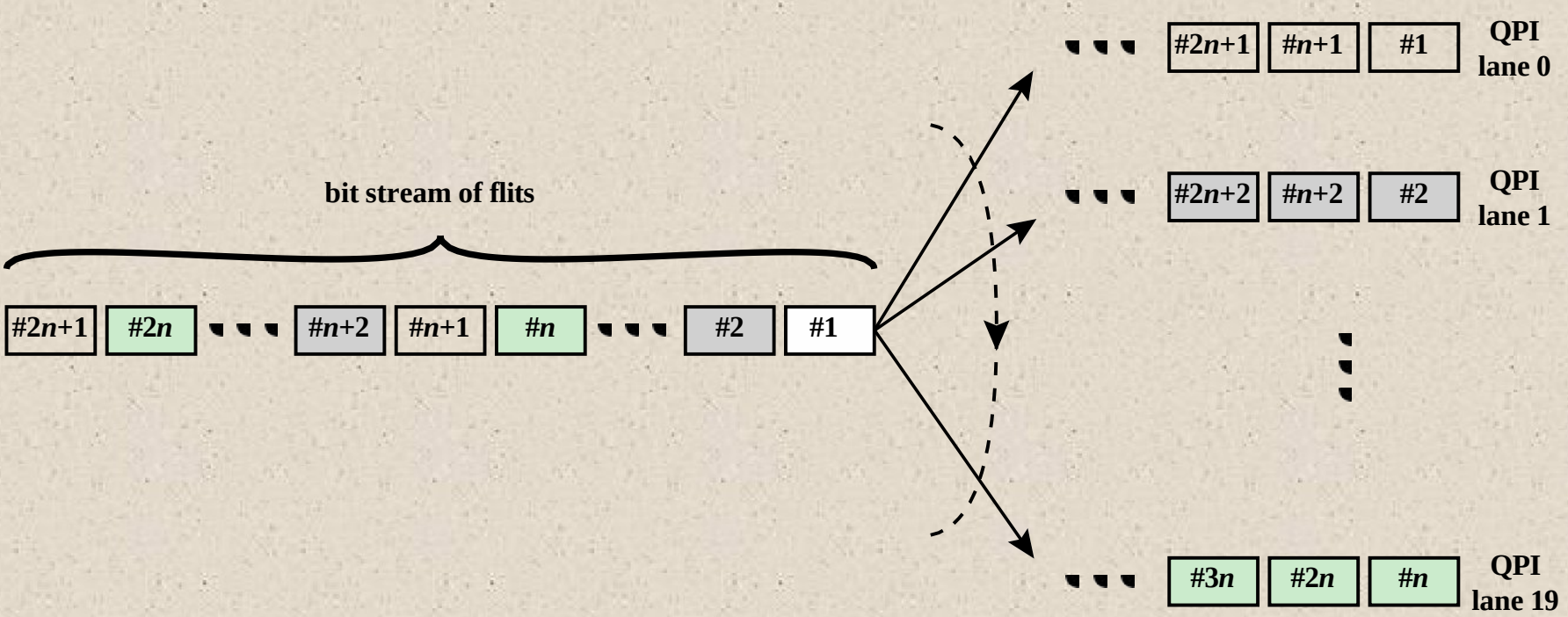


Figure 3.20 QPI Multilane Distribution

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

QPI Link Layer

- Performs two key functions: *flow control* and *error control*
  - Operate on the level of the flit (flow control unit)
  - Each flit consists of a 72-bit message payload and an 8-bit error control code called a *cyclic redundancy check* (CRC)
- Flow control function
  - Needed to ensure that a sending QPI entity does not overwhelm a receiving QPI entity by sending data faster than the receiver can process the data and clear buffers for more incoming data
- Error control function
  - Detects and recovers from bit errors, and so isolates higher layers from experiencing bit errors

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

QPI Routing and Protocol Layers

Routing Layer

- Used to determine the course that a packet will traverse across the available system interconnects
- Defined by firmware and describe the possible paths that a packet can follow

Protocol Layer

- Packet is defined as the unit of transfer
- One key function performed at this level is a cache coherency protocol which deals with making sure that main memory values held in multiple caches are consistent
- A typical data packet payload is a block of data being sent to or from a cache

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Peripheral Component Interconnect (PCI)

- A popular high bandwidth, processor independent bus that can function as a mezzanine or peripheral bus
- Delivers better system performance for high speed I/O subsystems
- PCI Special Interest Group (SIG)
  - Created to develop further and maintain the compatibility of the PCI specifications
- PCI Express (PCIe)
  - Point-to-point interconnect scheme intended to replace bus-based schemes such as PCI
  - Key requirement is high capacity to support the needs of higher data rate I/O devices, such as Gigabit Ethernet
  - Another requirement deals with the need to support time dependent data streams

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

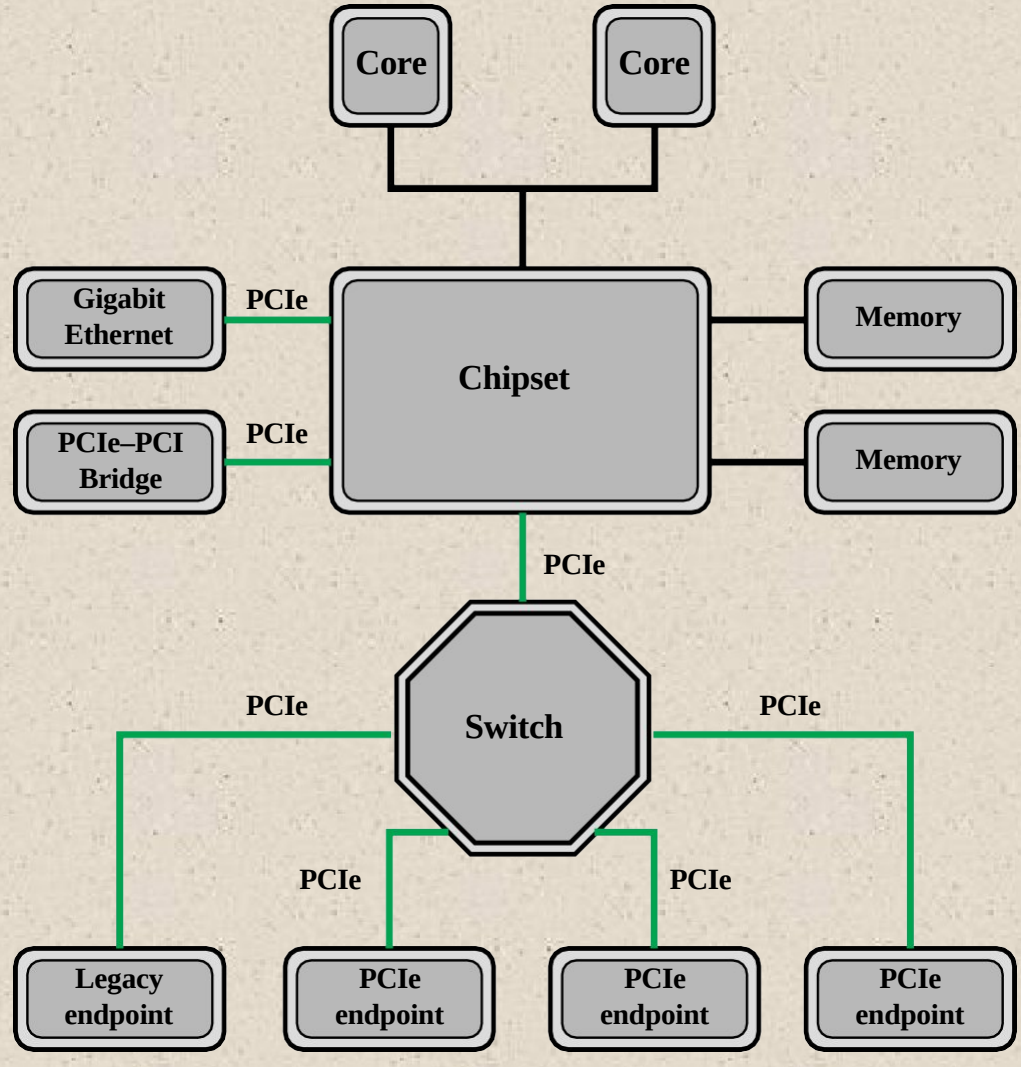


Figure 3.21 Typical Configuration Using PCIe

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

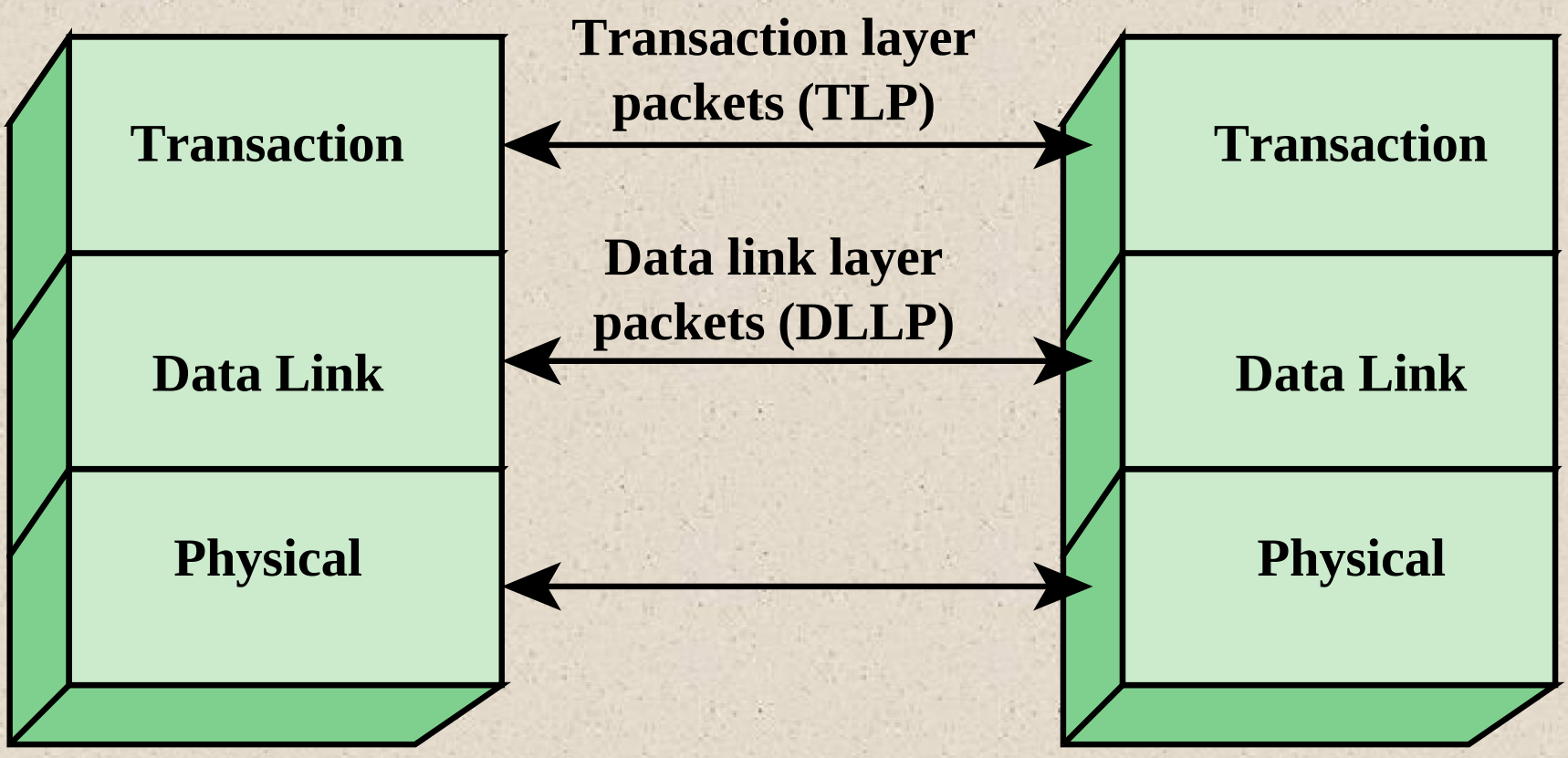


Figure 3.22 PCIe Protocol Layers

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

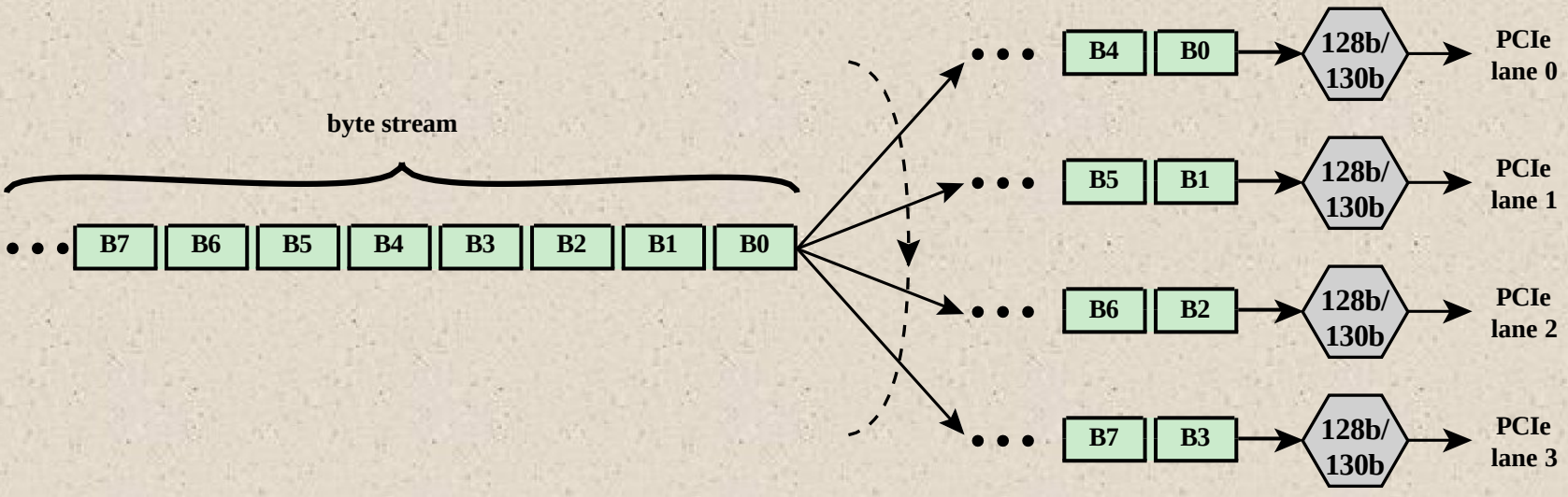


Figure 3.23 PCIe Multilane Distribution

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

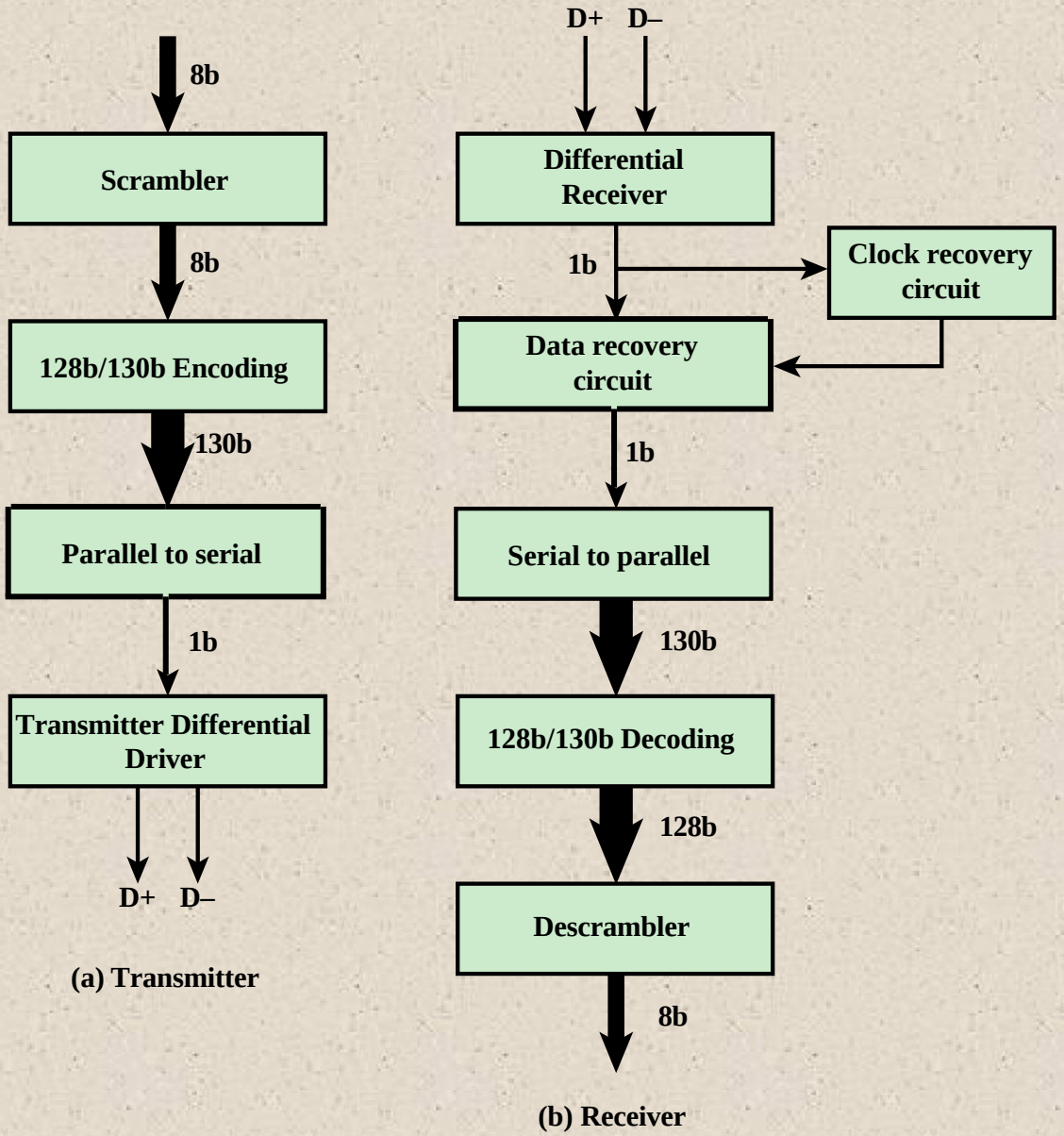


Figure 3.24 PCIe Transmit and Receive Block Diagrams

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

PCIe

Transaction Layer (TL)



- Receives read and write requests from the software above the TL and creates request packets for transmission to a destination via the link layer
- Most transactions use a *split transaction* technique
  - A request packet is sent out by a source PCIe device which then waits for a response called a *completion* packet
- TL messages and some write transactions are posted transactions (meaning that no response is expected)
- TL packet format supports 32-bit memory addressing and extended 64-bit memory addressing

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The TL supports four address spaces:

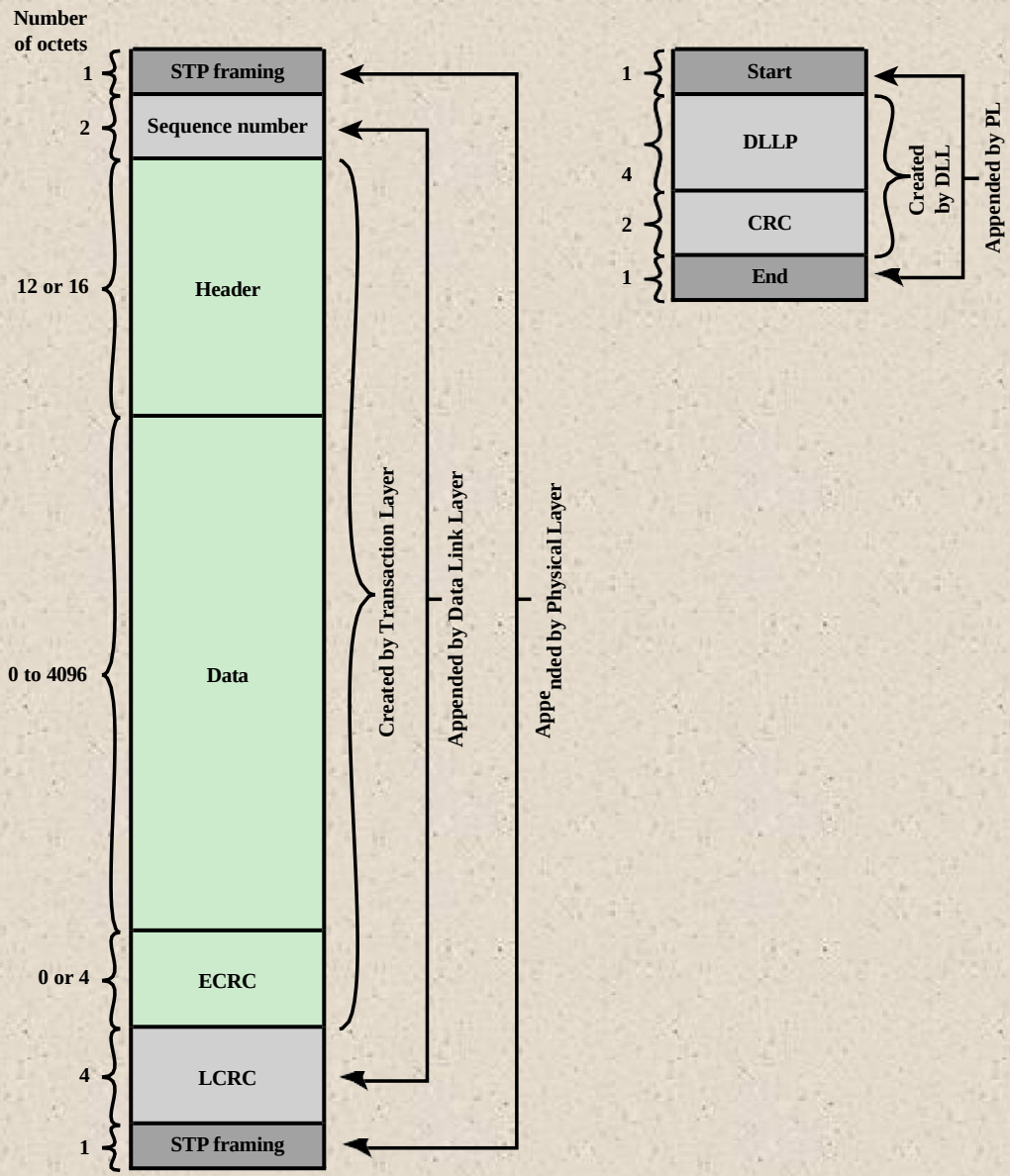
- Memory
  - The memory space includes system main memory and PCIe I/O devices
  - Certain ranges of memory addresses map into I/O devices
- Configuration
  - This address space enables the TL to read/write configuration registers associated with I/O devices
- I/O
  - This address space is used for legacy PCI devices, with reserved address ranges used to address legacy I/O devices
- Message
  - This address space is for control signals related to interrupts, error handling, and power management

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Table 3.2
PCIe TLP Transaction Types

Address Space	TLP Type	Purpose
Memory	Memory Read Request	Transfer data to or from a location in the system memory map.
	Memory Read Lock Request	
	Memory Write Request	
I/O	I/O Read Request	Transfer data to or from a location in the system memory map for legacy devices.
	I/O Write Request	
Configuration	Config Type 0 Read Request	Transfer data to or from a location in the configuration space of a PCIe device.
	Config Type 0 Write Request	
	Config Type 1 Read Request	
	Config Type 1 Write Request	
Message	Message Request	Provides in-band messaging and event reporting.
	Message Request with Data	
Memory, I/O, Configuration	Completion	Returned for certain requests.
	Completion with Data	
	Completion Locked	
	Completion Locked with Data	

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) Transaction Layer Packet (b) Data Link Layer Packet

Figure 3.25 PCIe Protocol Data Unit Format

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Summary

A Top-Level View of Computer Function and Interconnection

Chapter 3

- Computer components
- Computer function
  - Instruction fetch and execute
  - Interrupts
  - I/O function
- Interconnection structures
- Bus interconnection
- Point-to-point interconnect
  - QPI physical layer
  - QPI link layer
  - QPI routing layer
  - QPI protocol layer
- PCI express
  - PCI physical and logical architecture
  - PCIe physical layer
  - PCIe transaction layer
  - PCIe data link layer

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



William Stallings Computer Organization and Architecture 10th Edition

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Chapter 4

Cache Memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Location Internal (e.g. processor registers, cache, main memory) External (e.g. optical disks, magnetic disks, tapes) **Capacity** Number of words Number of bytes **Unit of Transfer** Word Block **Access Method** Sequential Direct Random Associative	**Performance** Access time Cycle time Transfer rate **Physical Type** Semiconductor Magnetic Optical Magneto-optical **Physical Characteristics** Volatile/nonvolatile Erasable/nonerasable **Organization** Memory modules

Table 4.1
Key Characteristics of Computer Memory Systems

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Characteristics of Memory Systems

- Location
  - Refers to whether memory is internal and external to the computer
  - Internal memory is often equated with main memory
  - Processor requires its own local memory, in the form of registers
  - Cache is another form of internal memory
  - External memory consists of peripheral storage devices that are accessible to the processor via I/O controllers
- Capacity
  - Memory is typically expressed in terms of bytes
- Unit of transfer
  - For internal memory the unit of transfer is equal to the number of electrical lines into and out of the memory module

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Method of Accessing Units of Data

- Sequential Access:
  - Whenever any access request arrives, the memory is searched from the beginning till the required data is found
  - Search begins at the first memory location and proceeds sequentially moving ahead one step at a time till the desired location is reached
  - Time taken to access any location is the variable.
  - Simple but slow technique
  - Tapes is an example of sequential access.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Method of Accessing Units of Data

- Random Access:
  - Data represent at any memory location can be accessed directly
  - Time taken to access any location is the same.
  - Faster way to retrieve data.
  - RAM is an example of this kind of access.
- Direct Access:
  - it's semi-random mode of operation in which data is stored in blocks /tracks which can be accessed randomly.
  - Unlike random access the time taken to access any location may not be the same
  - Used by magnetic disk

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

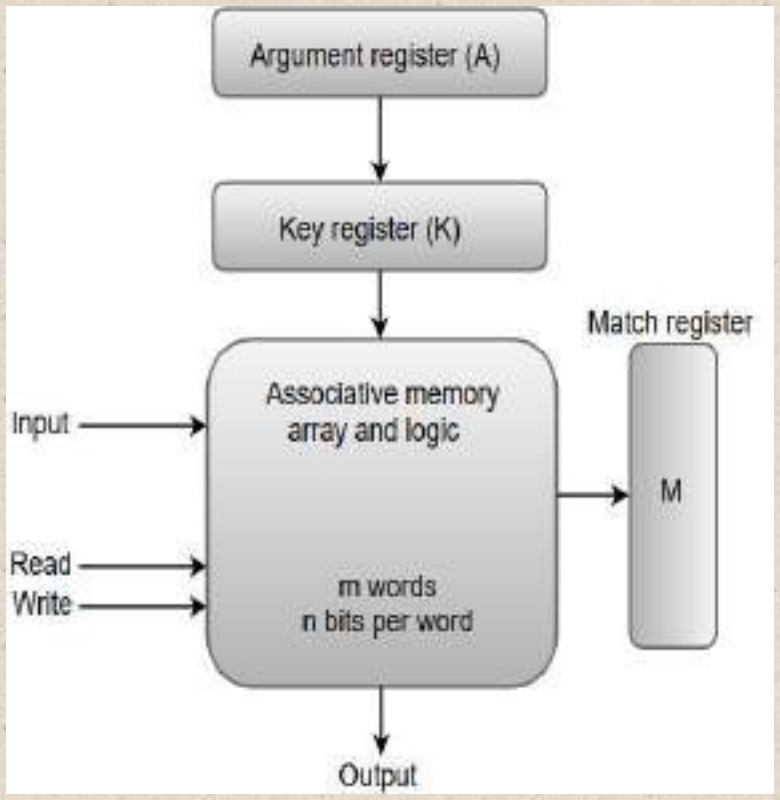
Method of Accessing Units of Data

- Associative:
  - When data is accessed by data content rather than data address, then the memory is referred to as associative memory or content addressable memory (CAM).
  - Data is stored at the very first empty location found in memory.
  - In associative memory when data is stored at a particular location then no address is stored along with it.
  - When the stored data need to be searched then only the key(i.e. data or part of data) is provided.
  - A sequential search is performed in the memory using the specified key to find out the matching key from the memory.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

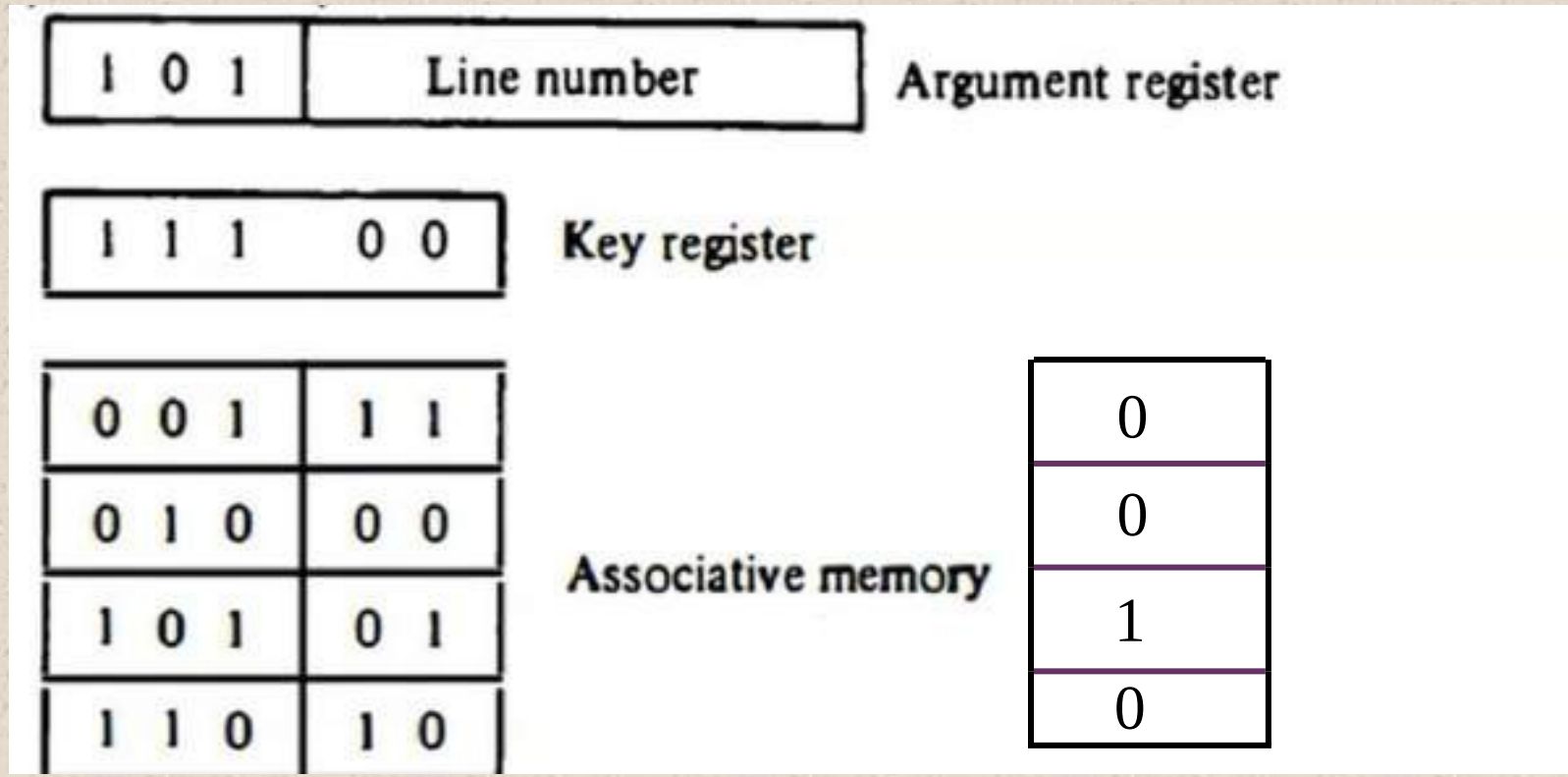
Block Diagram of Associative

- Argument Register (A): It contains the word to be searched.
- Key register (K): This specifies which part of the argument word needs to be compared with words in memory. If all bits in register are 1, the entire word should be compared else only the bits having 1 will be compared.
- Associative memory array: it contains the words which are to be compared with the argument word
- Match register (M): After the matching process the bits corresponding to matching words in match register are set to 1



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative example



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Method of Accessing Units of Data

Sequential access

- Memory is organized into units of data called records
- Access must be made in a specific linear sequence
- Access time is variable

Direct access

- Involves a shared read-write mechanism
- Individual blocks or records have a unique address based on physical location
- Access time is variable

Random access

- Each addressable location in memory has a unique, physically wired-in addressing mechanism
- The time to access a given location is independent of the sequence of prior accesses and is constant
- Any location can be selected at random and directly addressed and accessed
- Main memory and some cache systems are random access

Associative

- A word is retrieved based on a portion of its contents rather than its address
- Each location has its own addressing mechanism and retrieval time is constant independent of location or prior access patterns
- Cache memories may employ associative access

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Capacity and Performance:

The two most important characteristics of memory

Three performance parameters are used:

Access time (latency)

- For random-access memory it is the time it takes to perform a read or write operation
- For non-random-access memory it is the time it takes to position the read-write mechanism at the desired location

Memory cycle time

- Access time plus any additional time required before second access can commence
- Additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively
- Concerned with the system bus, not the processor

Transfer rate

- The rate at which data can be transferred into or out of a memory unit
- For random-access memory it is equal to 1/(cycle time)

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

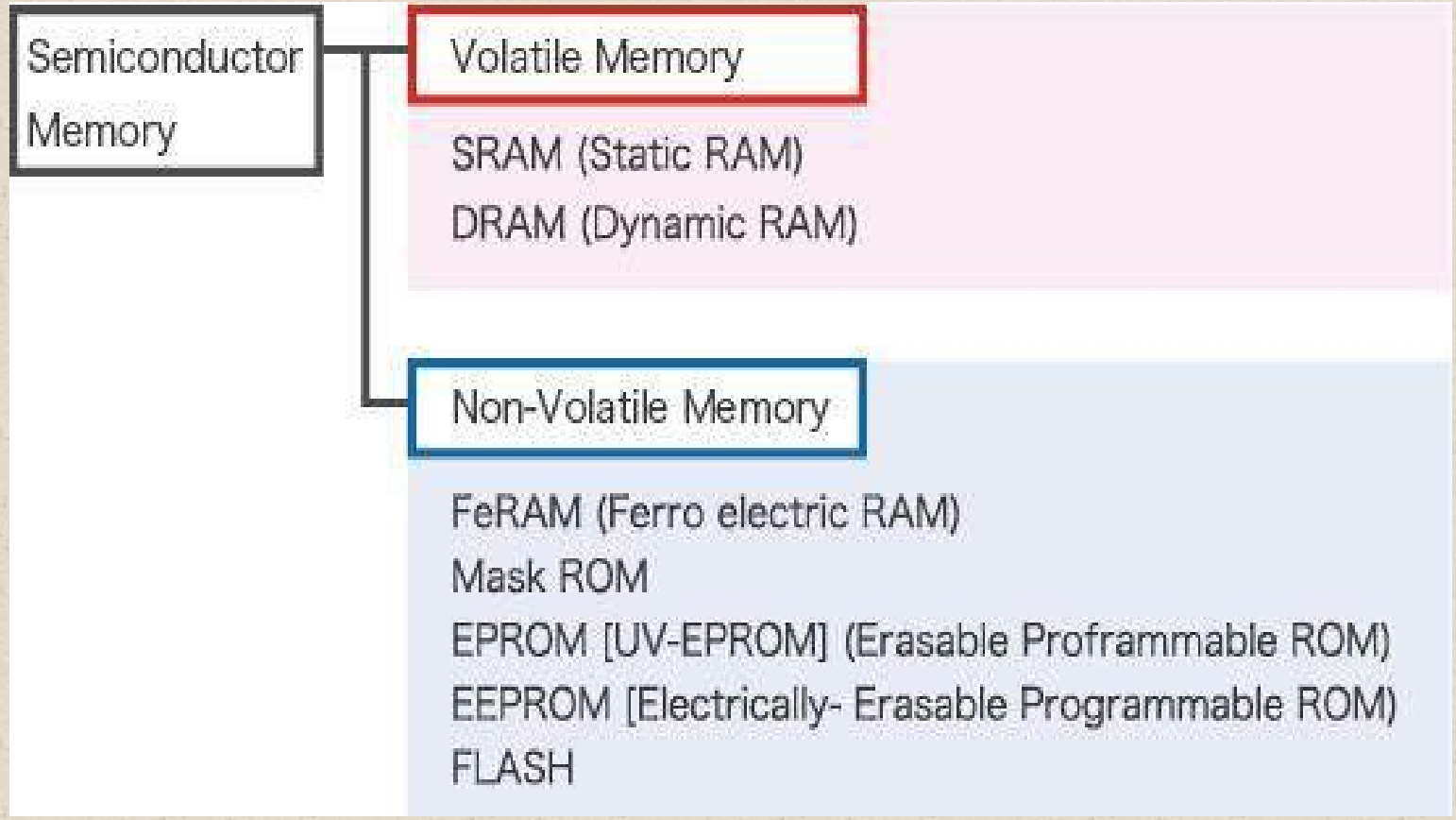
Memory



- The most common forms are:
  - Semiconductor memory
  - Magnetic surface memory
  - Optical
  - Magneto-optical
- Several physical characteristics of data storage are important:
  - Volatile memory
    - Information decays naturally or is lost when electrical power is switched off
  - Nonvolatile memory
    - Once recorded, information remains without deterioration until deliberately changed
    - No electrical power is needed to retain information
  - Magnetic-surface memories
    - Are nonvolatile
  - Semiconductor memory
    - May be either volatile or nonvolatile
  - Nonerasable memory
    - Cannot be altered, except by destroying the storage unit
    - Semiconductor memory of this type is known as read-only memory (ROM)
- For random-access memory the organization is a key design issue
  - Organization refers to the physical arrangement of bits to form words

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Semiconductor memory



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory Hierarchy

- Design constraints on a computer's memory can be summed up by three questions:
  - How much, how fast, how expensive
- There is a trade-off among capacity, access time, and cost
  - Faster access time, greater cost per bit
  - Greater capacity, smaller cost per bit
  - Greater capacity, slower access time
- The way out of the memory dilemma is not to rely on a single memory component or technology, but to employ a memory hierarchy

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

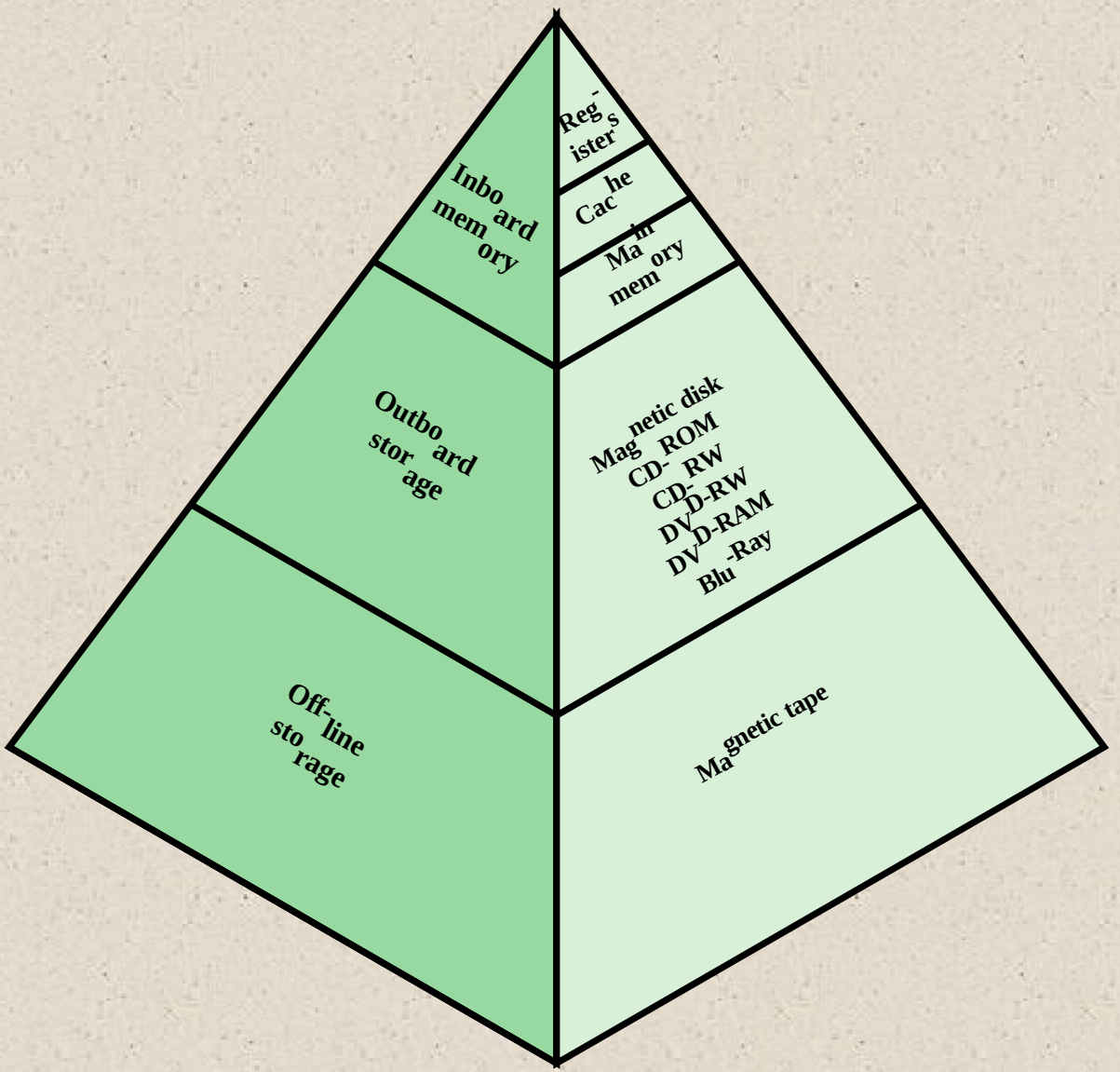


Figure 4.1 The Memory Hierarchy

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Memory

- The use of three levels exploits the fact that semiconductor memory comes in a variety of types which differ in speed and cost
- Data are stored more permanently on external mass storage devices
- External, nonvolatile memory is also referred to as **secondary** memory or **auxiliary** memory
- Disk cache
  - A portion of main memory can be used as a buffer to hold data temporarily that is to be read out to disk
  - A few large transfers of data can be used instead of many small transfers of data
  - Data can be retrieved rapidly from the software cache rather than slowly from the disk

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

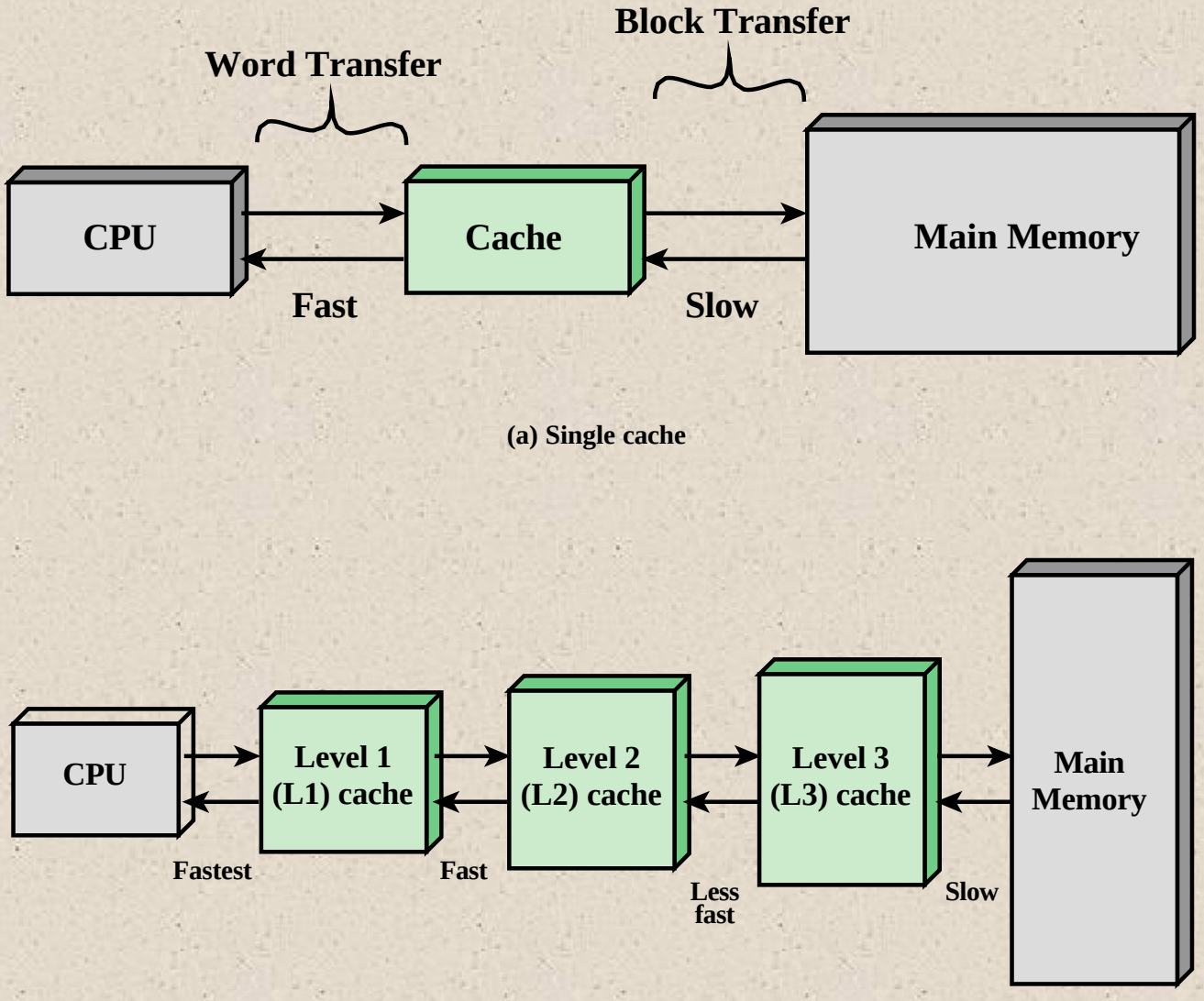


Figure 4.3 Cache and Main Memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

- In Figure 4.3a.The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block

- Figure 4.3b depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

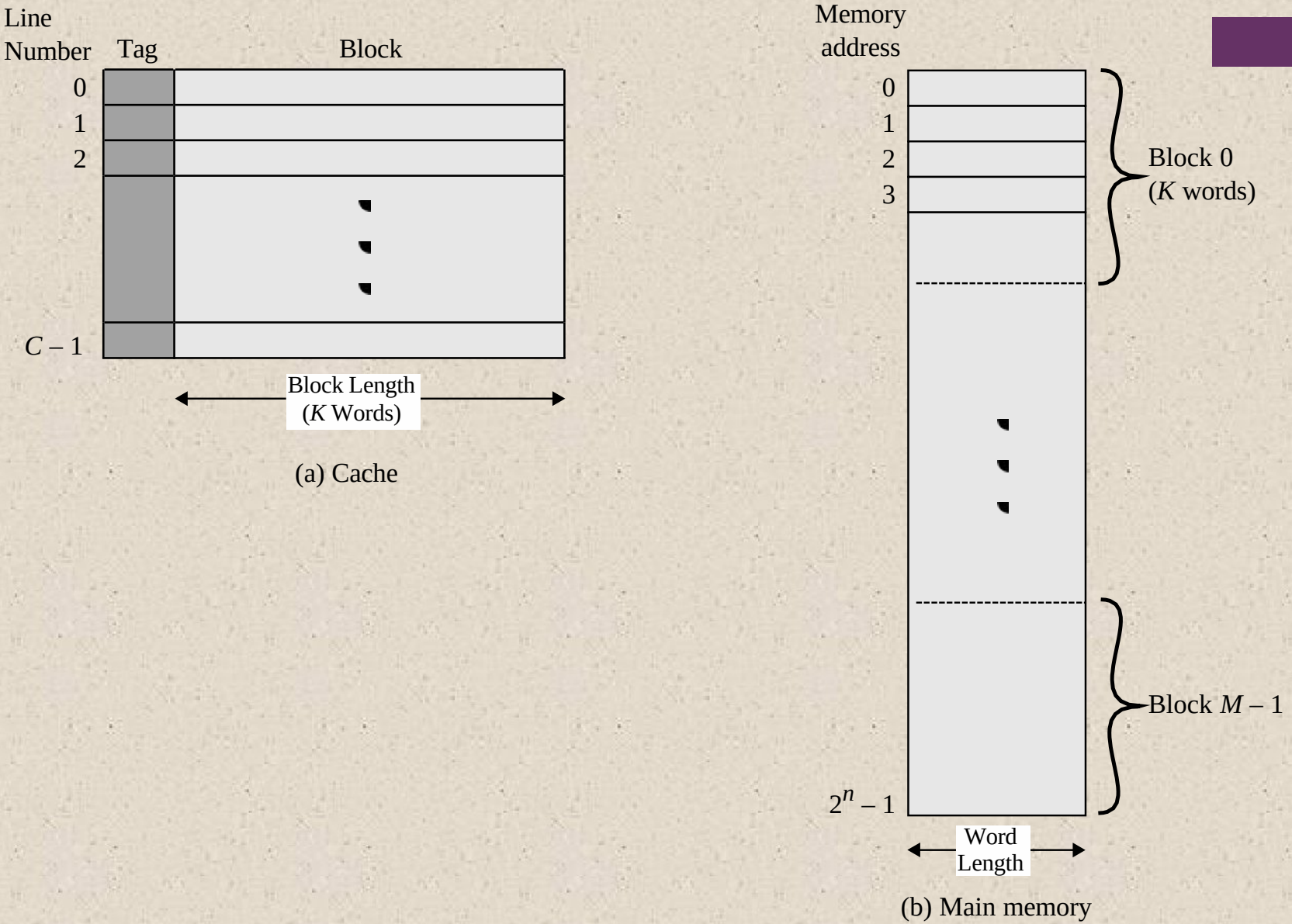


Figure 4.4 Cache/Main-Memory Structure

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

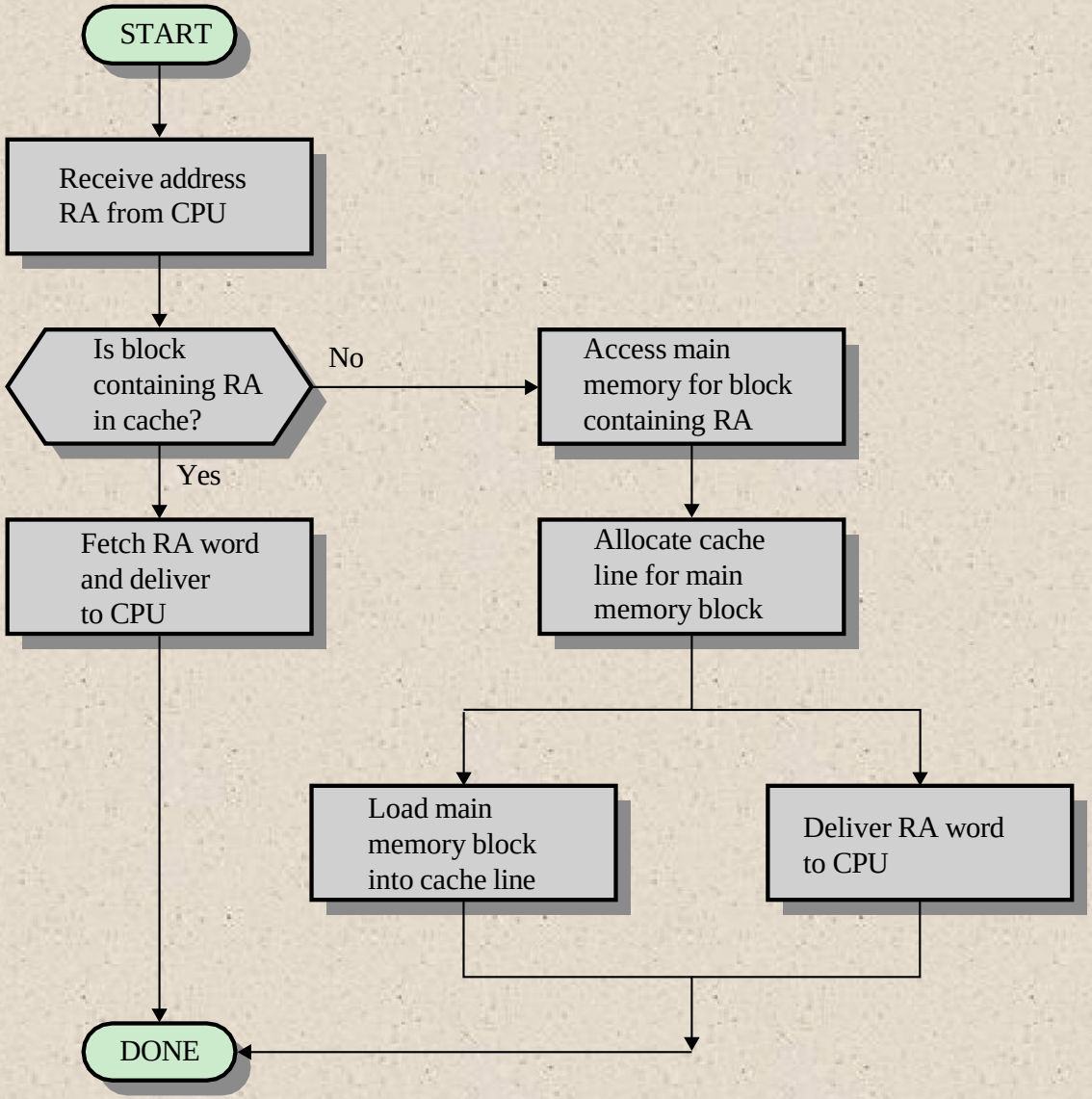


Figure 4.5 Cache Read Operation

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

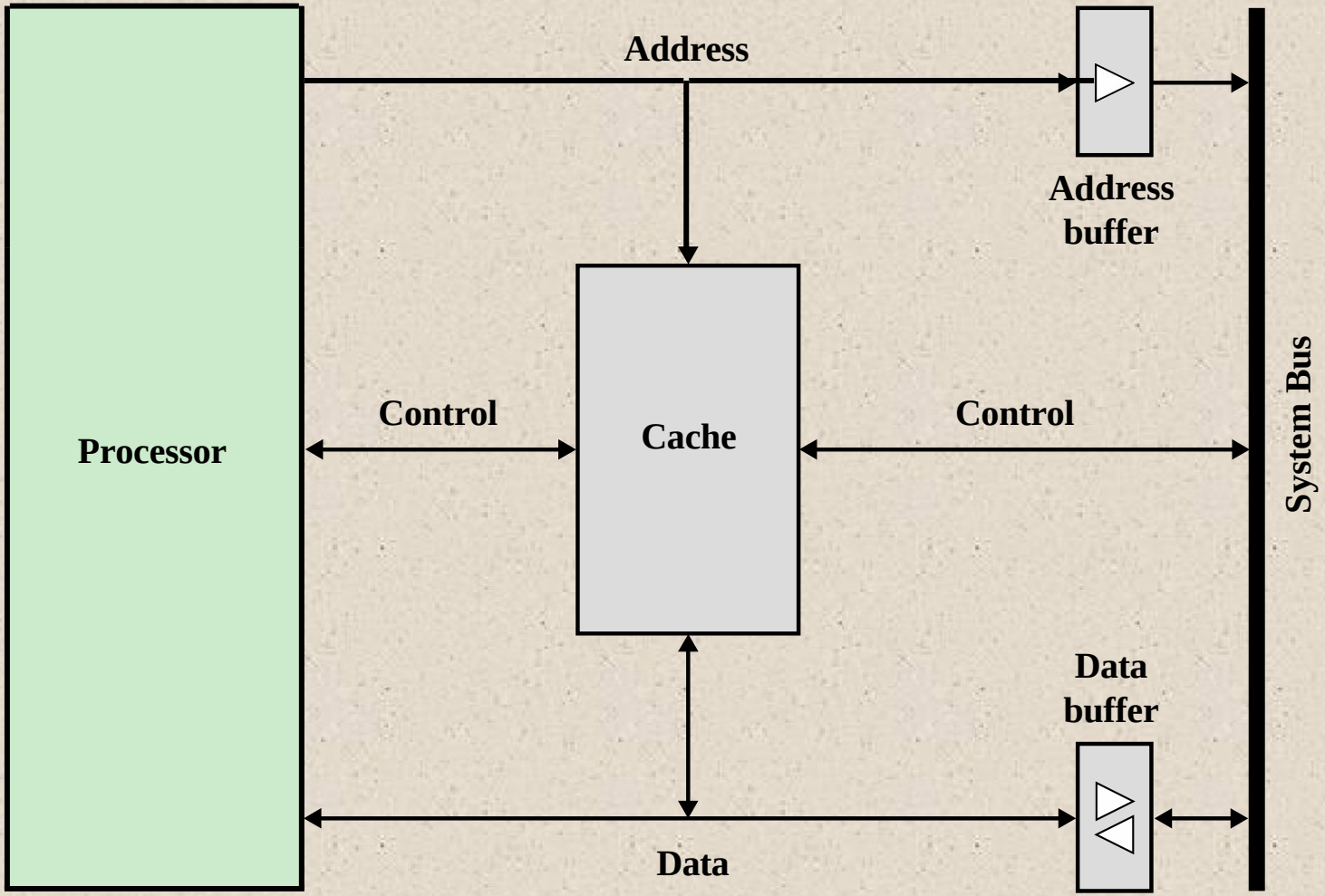


Figure 4.6 Typical Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Cache organization

- In figure 4.6, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from which main memory is reached.
- When a **cache hit** occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic.
- When a **cache miss** occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor.
- In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Cache Addresses	**Write Policy**
Logical	Write through
Physical	Write back
Cache Size	**Line Size**
Mapping Function	**Number of caches**
Direct	Single or two level
Associative	Unified or split
Set Associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

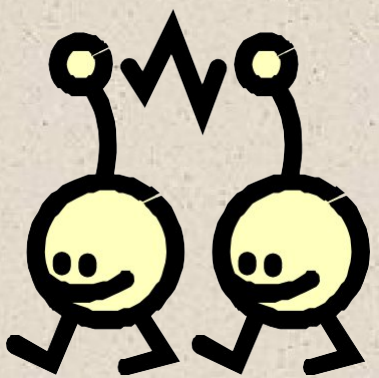
Table 4.2 Elements of Cache Design

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

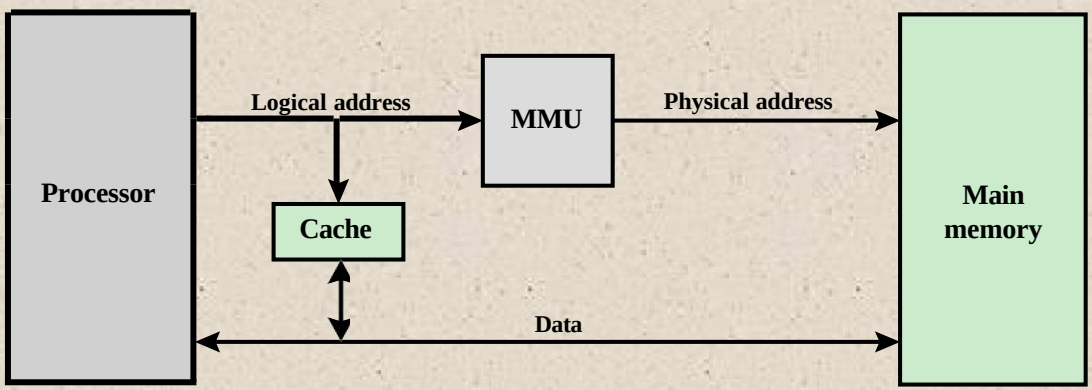
Cache Addresses

Virtual Memory

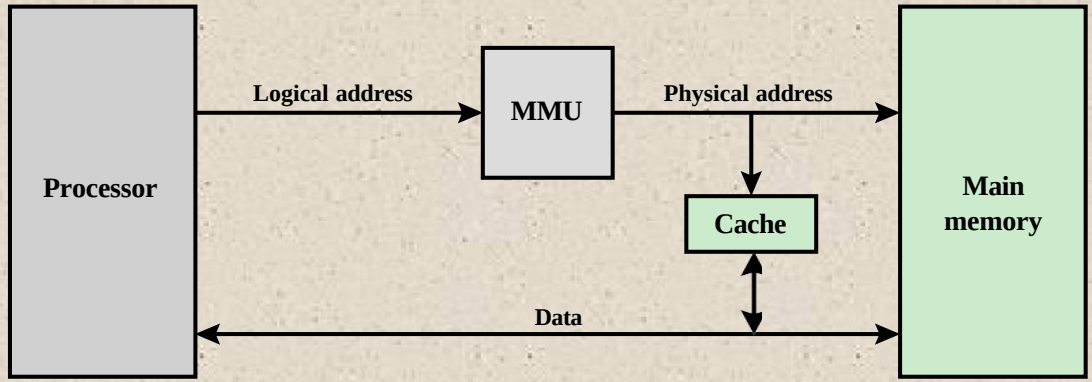
- Virtual memory
  - Facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available
  - When used, the address fields of machine instructions contain virtual addresses
  - For reads to and writes from main memory, a hardware memory management unit (MMU) translates each virtual address into a physical address in main memory



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.



(a) Logical Cache



(b) Physical Cache

Figure 4.7 Logical and Physical Caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Logical and physical caches

- When virtual addresses are used, the system designer may choose to place the cache between the processor and the MMU or between the MMU and main memory (Figure 4.7).
- A **logical cache,** also known as a **virtual cache,** stores data using **virtual addresses.** The processor accesses the cache directly, without going through the MMU.
- A physical cache stores data using main memory **physical addresses.**
- One obvious advantage of the logical cache is that **cache access speed** is faster than for a physical cache, because the cache can respond before the MMU performs an address translation.
- While the disadvantage is, each application sees a virtual memory that starts at address 0. Thus, the same virtual address in two different applications refers to two different physical addresses, The cache memory must therefore be completely flushed with each application context switch

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Cache size

- **Small enough** so that the overall average cost per bit is close to that of main memory alone and
- **Large enough** so that the overall average access time is close to that of the cache alone
- **The larger the cache**, the larger the number of gates involved in addressing the cache. The result is that large caches tend to be slightly slower than small ones – even when built with the same integrated circuit technology and put in the same place on chip and circuit board
- The **available chip and board area** also **limits** cache size
- Because the **performance of the cache** is very **sensitive** to the nature of the **workload**, it is **impossible** to arrive at a single '**optimum**' cache size

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Mapping Function

- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines
- Three techniques can be used:

1. Direct mapping :

- The simplest technique, known as **direct mapping**, maps each block of main memory into only one possible cache line.
- The mapping is expressed as

$$i = j \bmod m$$

Where : i = cache line number

j = main memory block number m

= number of lines in the cache

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

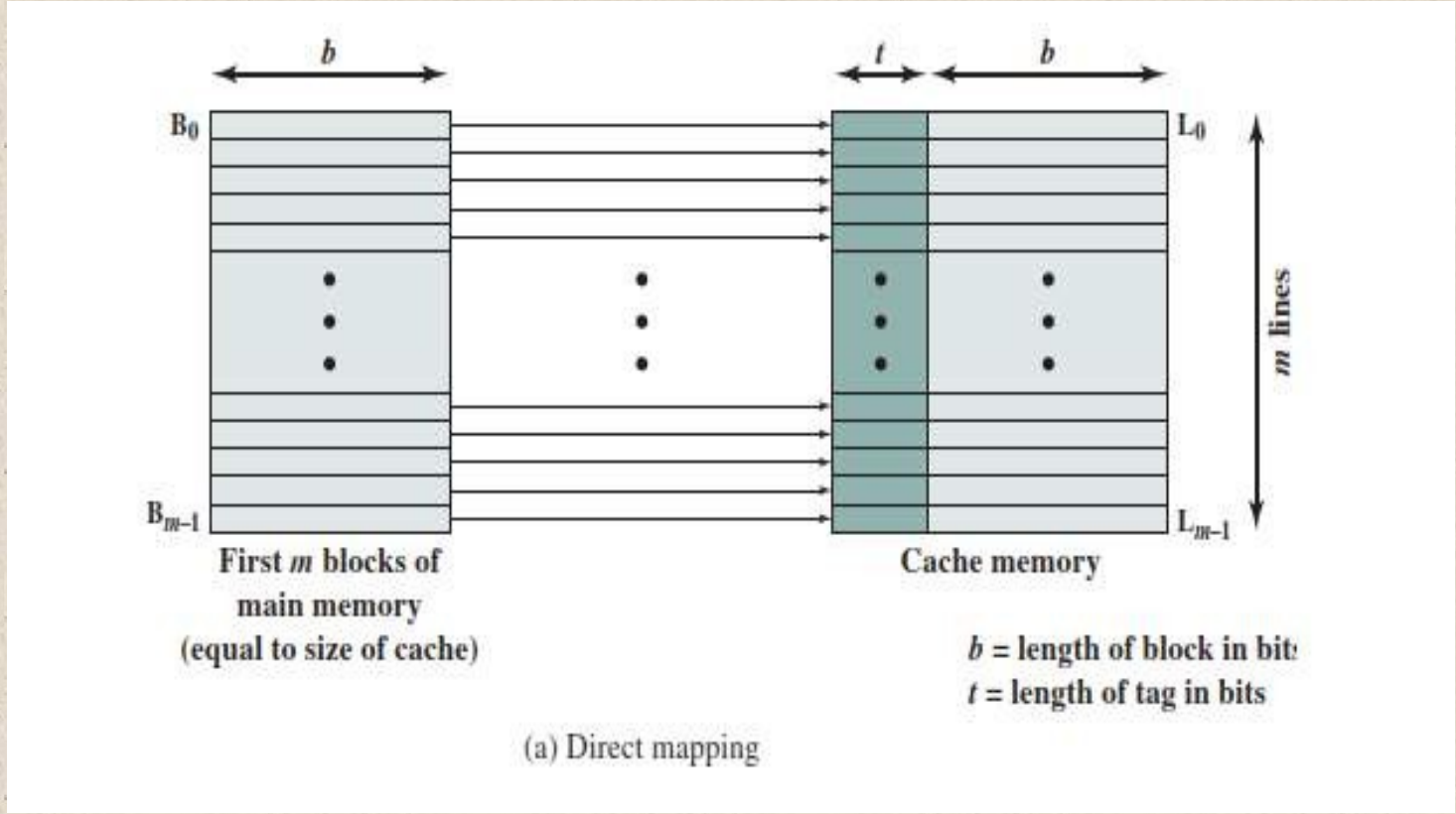


Figure 4.8 Mapping from Main Memory to Cache: Direct

Figure 4.8a shows the mapping for the first *m blocks of main memory. Each* block of main memory maps into one unique line of the cache. **The next *m blocks*** of main memory map into the cache in the same fashion; that is, block B_m *of main* memory maps into line L_0 of cache, block $_{Bm+1}$ *maps into line* L_1*, and so on.*

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Example

- If Cache size = 64 Kbytes and block size=4 bytes and memory size =16 Mbyte? Find total number of blocks in main memory and total cache lines within cache memory.

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Direct Mapping

- Each block of main memory maps to only one cache line
- Address is two parts
  - **W:** least significant bits identify **unique word**
  - **S:** Most significant bits identify **one memory block**
  - **S-bits** are split into

    s-r: Tag bits

    r:cache line

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Direct Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w} = 2^{s}$
- Number of lines in cache = m = 2^{r}
- Size of tag = (s – r) bits



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

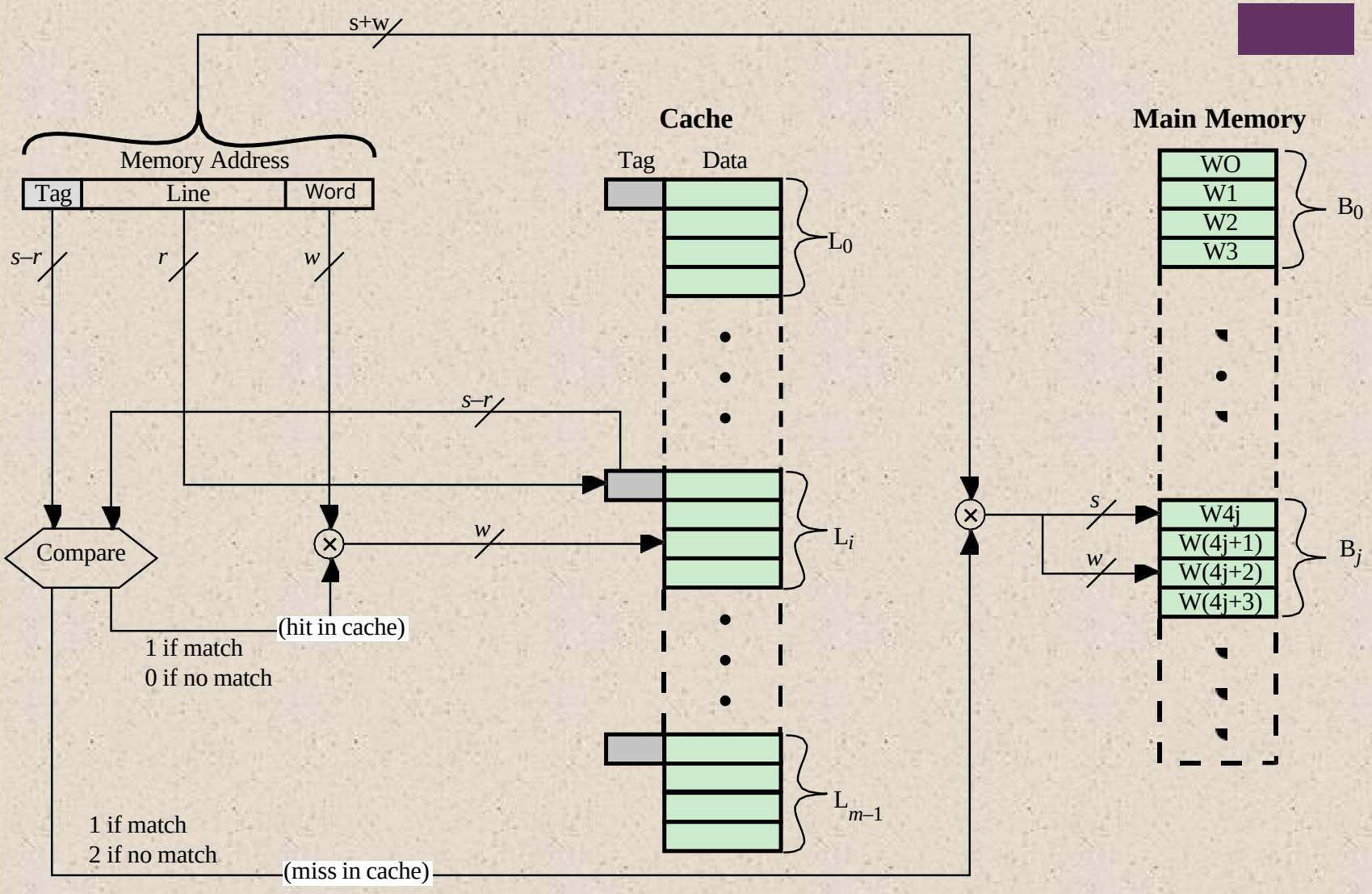


Figure 4.9 Direct-Mapping Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Figure 4.9 illustrates the general mechanism .
For purposes of cache access, each main memory address can be viewed as consisting of three fields.
The least significant w bits identify a unique word or byte within a block of main memory; in most contemporary machines, the address is at the byte level .
The remaining s bits specify one of the 2^s blocks of main memory.
The cache logic interprets these s bits as a tag of s - r bits (most significant portion) and a line field of r bits .
This latter field identifies one of the m = 2^r lines of the cache

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Advantage and disadvantage Direct mapping

- The direct mapping technique is simple and inexpensive to implement. Its
- main disadvantage is that there is a **fixed cache location** for any given block. Thus, if a program happens to reference words repeatedly from two different blocks that map into the same line, then the blocks will be continually swapped in the cache, and the hit ratio will be low (a phenomenon known as *thrashing).*

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Fully associative mapping

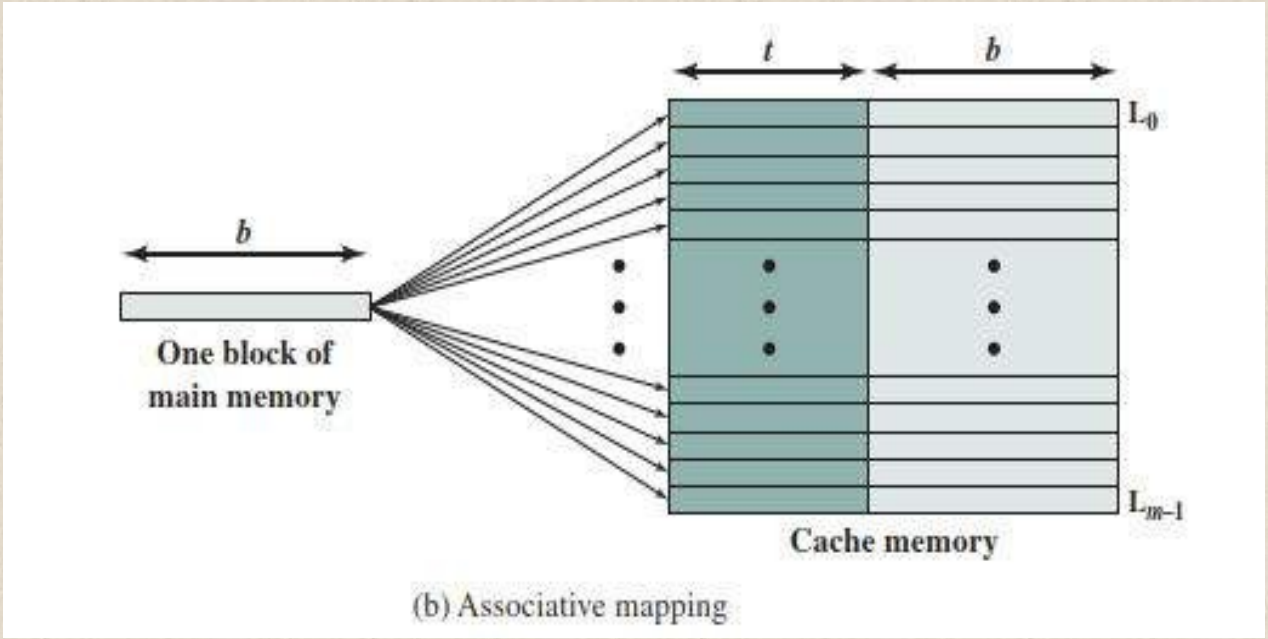


- Originally proposed as an approach to reduce the conflict misses of direct mapped caches without affecting its fast access time
- Also called victim cache
- Typical size is 4 to 16 cache lines
- Residing between direct mapped L1 cache and the next level of memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative mapping

- Overcome the disadvantage of direct mapping by **permitting each main memory block to be loaded into any line of the cache**



(b) Associative mapping

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative mapping

- The cache control logic interprets a memory address simply as a **Tag** and A **word** field.
- The Tag field uniquely identifies a block of main memory
- To determine whether a block is in the cache, the cache control logic must simultaneously examine every line's tag for a match



- Word is used for specific access within block
- 2 bit word 1 out of 4 words
- 4 bit word 1 out of 16 words

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

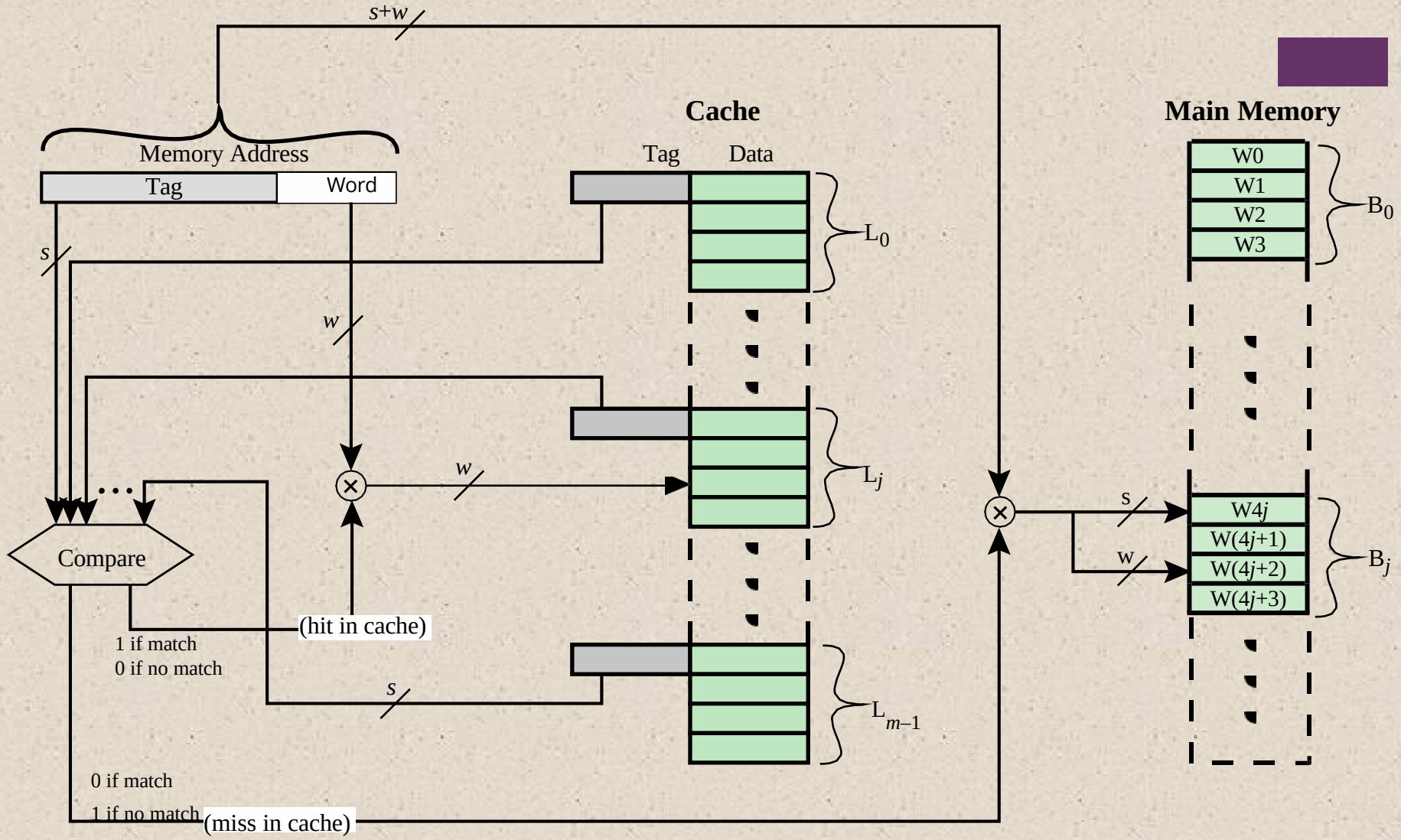


Figure 4.11 Fully Associative Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w} = 2^{s}$
- Number of lines in cache = undetermined
- Size of tag = s bits



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Example

- Using mapping If Cache size = 64 Kbytes and block size=4 bytes and memory size =16 Mbyte? Find total number of blocks in main memory, and the content of memory address
- Sol

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Associative Mapping Pros & Cons

- With associative mapping there is **flexibility** as to which block to replace when a new block is read into the cache.

Note: Replacement algorithms are designed to maximize the hit ratio.

Complex circuitry required to examine the tags of all cache lines in parallel

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping

- Compromise that exhibits the strengths of both the direct and associative approaches while reducing their disadvantages
- The cache consists of several sets, each of which consists of several lines

$$m = v \times k$$

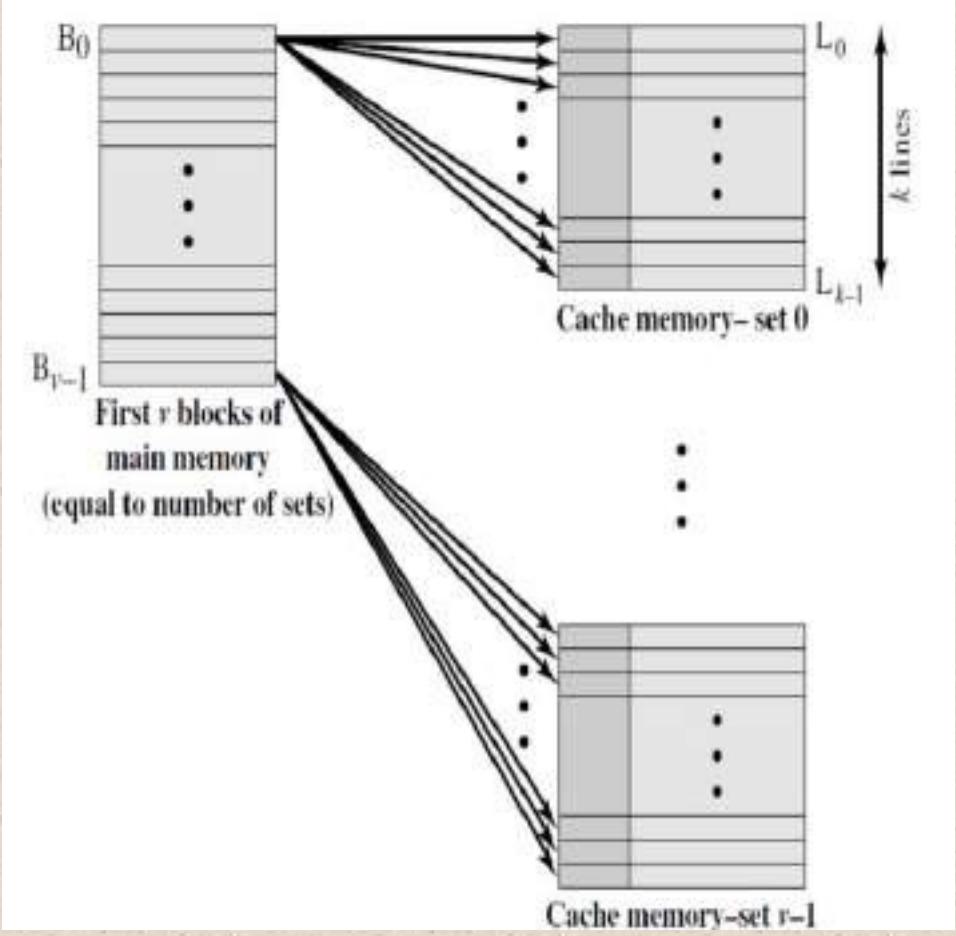
$$i = j \ modulo \ v$$

- i cache set number
- j main memory block number
- m number of lines in the cache
- v number of sets
- k number of lines in each set

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping

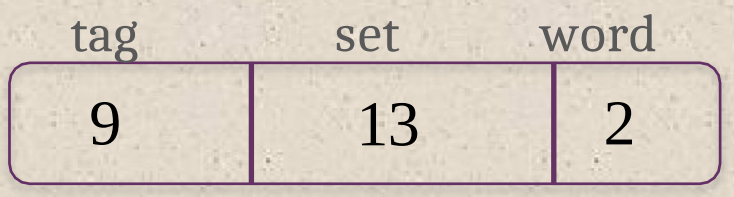
- Each word maps into all the cache lines in a specific set
- Set-associative cache can be Physically implemented as associative caches



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping

- Cache size : 64 Kbytes.
- Block size: 4 bytes
- Cache is organized as 16 $K = 2^{14}$ **lines** of 4 bytes each.
- Memory Size: 16 Mbytes, with each byte directly addressable by a 24-bit address ($2^{24} = 16\ M$).
- Main memory: (2^{22} =4M)blocks of 4 bytes each
- each set contain 2 line then
- # of set(v)= 2^{14} / 2^{1} =2^{13} = 8 K
- Size of tag= (s-d) 22-13 = 9



© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping

- Memory address as three fields: **Tag, Set, and Word.**
- **The d set bits specify one of $v = 2^d$ sets**
- **The s bits of the Tag and Set fields specify one of the 2^s blocks of main memory.**
- **Number of sets(v) can be calculate by dividing cache size into k(number of line in each set)**

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

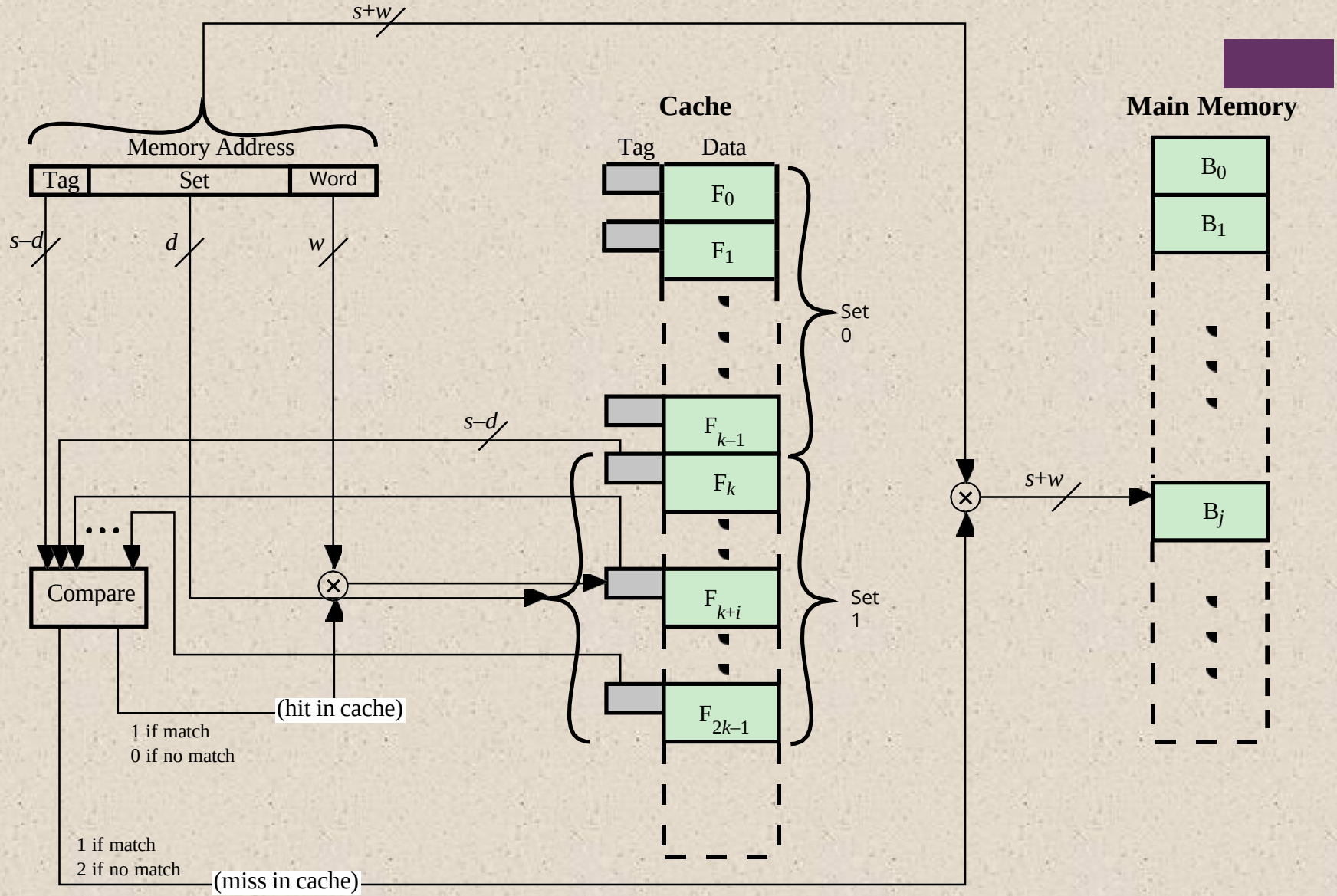


Figure 4.14 *k*-Way Set Associative Cache Organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Figure 4.14 illustrates the cache control logic. With **fully associativ**e mapping, the tag in a memory address is quite large and must be compared to the tag of every line in the cache. With ***k-way set-associative*** *mapping, the tag in a memory address is* much smaller and is only compared to the *k tags within a single set*

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping Summary

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^{w} words or bytes
- Number of blocks in main memory = $2^{s+w}/2^{w}=2^{s}$
- Number of lines in set = k
- Number of sets = v = 2^{d}
- Number of lines in cache = m=kv = k * 2^{d}
- Size of cache = $k * 2^{d+w}$ words or bytes
- Size of tag = (s – d) bits



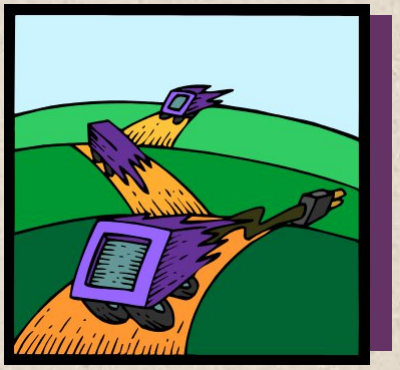
© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Set Associative Mapping

- In the extreme case of $v = m$, $k = 1$, the set-associative technique reduces to direct mapping
- for $v = 1, k = m$, it reduces to associative mapping.
- The use of two lines per set (v = m/2, k = 2) is the most common set-associative organization

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Replacement Algorithms



- Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced
- For direct mapping there is only one possible line for any particular block and no choice is possible
- For the associative and set-associative techniques a replacement algorithm is needed
- To achieve high speed, an algorithm must be implemented in hardware

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

The most common replacement algorithms are:

- Least recently used (LRU)
  - Most effective
  - Replace that block in the set that has been in the cache **longest with no reference to it**
  - Because of its simplicity of implementation, LRU is the most popular replacement algorithm
- First-in-first-out (FIFO)
  - Replace that block in the set that has been in the cache longest
  - Easily implemented as a round-robin or **circular buffer technique**
- Least frequently used (LFU)
  - Replace that block in the set that has experienced the **fewest references**
  - Could be implemented by associating a counter with each line

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Write Policy

When a block that is resident in the cache is to be replaced there are two cases to consider:

If the old block in the cache has not been altered then it may be overwritten with a new block without first writing out the old block

If at least one write operation has been performed on a word in that line of the cache then main memory must be updated by writing the line of cache out to the block of memory before bringing in the new block

There are two problems to contend with:

More than one device may have access to main memory

A more complex problem occurs when multiple processors are attached to the same bus and each processor has its own local cache - if a word is altered in one cache it could conceivably invalidate a word in other caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Write Through and Write Back

- Write through
  - Simplest technique
  - All write operations are made to main memory as well as to the cache
  - The main disadvantage of this technique is that it generates substantial memory traffic and may create a bottleneck
- Write back
  - Minimizes memory writes
  - Updates are made only in the cache
  - When an update occurs, a dirty bit, or use bit, associated with the line is set. Then, when a block is replaced, it is written back to main memory if and only if the dirty bit is set
  - The problem with write back is "cache coherent" different copies for block between cache and main memory

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.

Summary

Chapter 4

Cache Memory

- Computer memory system overview
  - Characteristics of Memory Systems
  - Memory Hierarchy
- Cache memory principles
- Pentium 4 cache organization

- Elements of cache design
  - Cache addresses
  - Cache size
  - Mapping function
  - Replacement algorithms
  - Write policy
  - Line size
  - Number of caches

© 2016 Pearson Education, Inc., Hoboken, NJ. All rights reserved.